

INFORME TÉCNICO: INSTRUMENTUM

- Sección: 010D, Grupo 2
 - Integrantes:
 - Victoria Bustos
 - Oscar Tavolari
- Profesor: Eduardo Baeza

1. Introducción.....	5
2. Objetivos.....	6
2.1 Objetivo General.....	6
2.2 Objetivos Específicos.....	6
3. Descripción del Proyecto.....	7
3.1 Problemática.....	7
3.2 Solución Propuesta.....	7
3.3 Funcionamiento General.....	8
3.4 Módulos del Sistema.....	8
4. Arquitectura del Sistema.....	10
4.1 Arquitectura General.....	10
4.1.1 API Gateway.....	11
4.1.3 Microservicios.....	11
5. Modelo de Base de Datos.....	14
5.1 Banda.....	14
5.2 Usuario.....	15
5.3 Equipo.....	15
5.4 Marca.....	16
5.5 Categoría.....	16
5.6 Especificación Instrumento.....	16
5.7 Especificación Electrónica.....	16
5.8 Mantenimiento.....	17
5.9 Canción.....	17
5.10 Equipo_Canción.....	17
5.11 Evento.....	17
5.12 Gira.....	18
5.13 Parada de Gira.....	18
5.14 Producto Merchandising.....	18
5.15 Venta Merchandising.....	18
5.16 Contenedor.....	19
5.17 Contenedor_Equipo.....	19
5.18 Transacción.....	19
Resumen del modelo.....	19
6. Servicio de Autenticación.....	20
6.1 Implementación mediante Token.....	20
6.2 Flujo de autenticación.....	21
6.3 Configuración del servicio de autenticación.....	26
Seguridad y JWT.....	26
JSON Web Token (JWT).....	26
jjwt-impl:.....	27
jjwt-jackson.....	27
Spring Boot Web: (spring-boot-starter-web).....	27
Dependencias de Base de Datos.....	28

MySQL Connector (mysql-connector-j).....	28
6.3.2 Clases principales del servicio de autenticación.....	29
SecurityConfig.....	29
AuthService.....	31
JwtService.....	32
AuthController.....	33
UsuarioRepository.....	34
Modelos del servicio de autenticación.....	34
Entidad Usuario.....	34
Relación Usuario - Rol.....	36
Entidad Rol.....	37
DTO de autenticación (AuthRequest).....	37
7. Implementación por Capas.....	38
Model.....	39
Repository.....	40
Responsabilidad.....	40
UsuarioRepository.....	41
Responsabilidades:.....	41
Métodos implementados:.....	41
BandaRepository.....	42
Responsabilidades:.....	42
Método implementado:.....	42
Métodos heredados de JpaRepository:.....	42
Service.....	43
Responsabilidades principales.....	43
Flujo general de operaciones.....	43
Registro de usuario.....	43
Consulta de usuarios.....	43
Actualización de usuario.....	43
Eliminación de usuario.....	43
Controller.....	46
UsuarioController.....	46
Endpoints principales.....	46
Crear usuario.....	46
Listar usuarios.....	47
Obtener usuario por ID.....	48
Actualizar usuario.....	48
Usuarios por banda.....	49
Eliminar usuario.....	50
Manejo de excepciones.....	50
Base de datos.....	51
Endpoints principales.....	51
Crear banda.....	51
Listar bandas.....	52

Obtener banda por ID.....	53
Actualizar banda.....	54
Eliminar banda.....	55
Manejo de excepciones.....	55
8. Comunicación entre Microservicios.....	56
9. Swagger.....	56
¿Qué es?.....	56
¿En qué consiste?.....	56
1. Microservicio de Eventos:.....	57
2. Microservicio de Logística.....	58
3. Microservicio de Especificaciones (Specs).....	60
10. Testing.....	61
10.1 ¿Qué es el testing?.....	61
10.3 Documentación de Pruebas Implementadas (Microservicios).....	63
1. Microservicio: Finanzas.....	63
2. Microservicio: Giras.....	64
3. Microservicio: Logística.....	66
4. Microservicio: Merchandising.....	66
5. Microservicio: Mantenimiento.....	68
6. Microservicio: Inventario.....	68
Resultados de Ejecución de los Tests Global.....	70
11. Conclusiones.....	71
11.1 Arquitectura.....	71
11.2 Modularidad.....	71
11.3 Seguridad.....	71
11.4 Mantenibilidad.....	72
11.5 Escalabilidad.....	72
12. Bibliografía.....	72

1. Introducción

El presente documento tiene como finalidad describir el desarrollo e implementación del proyecto Instrumentum, una plataforma orientada a la administración y gestión de equipamiento musical. El sistema permite controlar de manera organizada los diferentes elementos asociados a una agrupación musical, incluyendo la administración de usuarios, instrumentos, especificaciones técnicas, mantenimiento, inventario, logística, giras, eventos, finanzas y productos de merchandising.

Instrumentum fue desarrollado utilizando una arquitectura basada en microservicios, permitiendo dividir el sistema en componentes independientes y especializados. Cada microservicio representa un módulo funcional del negocio, facilitando la escalabilidad, mantenimiento y evolución del sistema. La implementación fue realizada utilizando Spring Boot como framework principal para el desarrollo del backend y MySQL como sistema gestor de base de datos para el almacenamiento de la información.

La solución contempla una arquitectura distribuida compuesta por un API Gateway, encargado de centralizar las solicitudes realizadas por los clientes y comunicarla con los diferentes servicios internos, además de múltiples microservicios responsables de funcionalidades específicas como autenticación, usuarios, inventario, mantenimiento, logística, giras, eventos, finanzas y merchandising.

Dentro del sistema se incorpora un mecanismo de seguridad basado en autenticación mediante JWT (JSON Web Token), permitiendo proteger los recursos y controlar el acceso a las operaciones disponibles. Este enfoque permite validar las solicitudes mediante tokens de seguridad, evitando el uso de sesiones tradicionales y mejorando la protección de la información.

Para la gestión del desarrollo del proyecto se utilizó Git como sistema de control de versiones y GitHub como plataforma de almacenamiento y colaboración del código fuente. El desarrollo fue realizado mediante un flujo de trabajo basado en ramas independientes para cada desarrollador, permitiendo separar los avances, reducir conflictos y mantener una organización adecuada del código. Posteriormente, los cambios fueron integrados mediante el proceso de combinación de ramas, manteniendo una versión centralizada y actualizada del proyecto.

A lo largo del documento se presenta la descripción general del proyecto, abordando la problemática identificada y la solución propuesta, además del funcionamiento de los módulos principales. Posteriormente se explica la arquitectura utilizada, el modelo de datos diseñado, la implementación de los servicios mediante capas (Model, Repository, Service y Controller), la comunicación entre microservicios, documentación mediante Swagger y las pruebas realizadas para verificar el correcto funcionamiento del sistema.

Finalmente, se presentan las conclusiones obtenidas durante el desarrollo, destacando las decisiones técnicas utilizadas, las herramientas empleadas y los beneficios de aplicar una arquitectura basada en microservicios junto con buenas prácticas de desarrollo colaborativo para la creación de una plataforma de gestión musical.

2. Objetivos

El desarrollo del proyecto Instrumentum tiene como propósito implementar una plataforma tecnológica orientada a la gestión integral de equipamiento musical, aplicando principios de arquitectura distribuida y buenas prácticas de desarrollo de software.

Los objetivos definidos buscan establecer una solución modular, segura y escalable que permita administrar la información de los distintos procesos involucrados, facilitando la gestión de usuarios, instrumentos, inventario, eventos, mantenimiento, giras y demás componentes del sistema.

2.1 Objetivo General

Desarrollar una plataforma basada en una arquitectura de microservicios que permita administrar la información relacionada con usuarios, inventario, eventos, giras, mantenimiento y otros módulos del sistema Instrumentum, proporcionando una solución modular, escalable y segura mediante el uso de Spring Boot, MySQL y tecnologías complementarias para la comunicación, autenticación y documentación de servicios.

2.2 Objetivos Específicos

- **Implementar una arquitectura basada en microservicios:** Separando las funcionalidades principales del sistema en servicios independientes, permitiendo una mejor organización del código, mantenimiento y escalabilidad del proyecto.
- **Desarrollar operaciones CRUD (Create, Read, Update, Delete)** para cada dominio del sistema, permitiendo la creación, consulta, actualización y eliminación de información correspondiente a cada módulo funcional.
- **Diseñar e implementar un API Gateway** como punto central de comunicación entre los clientes y los microservicios internos, facilitando la administración de solicitudes y el acceso uniforme a los servicios disponibles.

- **Implementar un sistema de autenticación basado en JWT (JSON Web Token)**, permitiendo validar la identidad de los usuarios y proteger los endpoints que requieren autorización.
- **Documentar los servicios mediante Swagger/OpenAPI**, proporcionando una interfaz que permita visualizar, probar y comprender los endpoints disponibles en cada microservicio.
- **Implementar pruebas unitarias** para verificar el correcto funcionamiento de las funcionalidades principales, asegurando la calidad y estabilidad del código desarrollado.
- **Persistir la información utilizando MySQL**, diseñando un modelo de datos capaz de almacenar y gestionar la información generada por los diferentes módulos del sistema.
- **Aplicar buenas prácticas de desarrollo colaborativo utilizando Git y GitHub**, permitiendo administrar versiones del código fuente mediante ramas de trabajo independientes e integración de cambios entre desarrolladores.

3. Descripción del Proyecto

3.1 Problemática

Las bandas musicales y los equipos técnicos administran diariamente una gran cantidad de información relacionada con instrumentos, equipos electrónicos, repertorio, presentaciones, mantenimiento, logística y recursos financieros. En muchos casos, esta información se gestiona de forma manual mediante planillas de cálculo, documentos de texto o aplicaciones independientes, lo que dificulta mantener un registro actualizado y centralizado.

Esta situación puede generar problemas como la pérdida de información, duplicidad de registros, dificultad para conocer el estado del equipamiento, falta de control sobre los mantenimientos preventivos, desorganización en la planificación de giras y eventos, además de una limitada trazabilidad de las ventas de merchandising y de las transacciones económicas de la banda.

Adicionalmente, la ausencia de una plataforma integrada dificulta el acceso oportuno a la información por parte de los distintos integrantes de la agrupación, afectando la coordinación entre músicos, técnicos y administradores.

3.2 Solución Propuesta

Como respuesta a esta problemática se desarrolló Instrumentum, una plataforma orientada a la gestión integral de bandas musicales, cuyo propósito es centralizar la administración de la información técnica, operativa y administrativa en un único sistema.

La aplicación fue diseñada utilizando una arquitectura basada en microservicios, permitiendo dividir el sistema en componentes especializados e independientes, facilitando su mantenimiento, escalabilidad y evolución. De esta forma, cada funcionalidad se encuentra desacoplada del resto del sistema, permitiendo realizar modificaciones o incorporar nuevas características con un menor impacto sobre los demás componentes.

Asimismo, la plataforma incorpora mecanismos de autenticación mediante tokens, documentación de servicios web, persistencia de datos utilizando una base de datos relacional y una estructura organizada por capas que favorece la reutilización del código y la separación de responsabilidades.

3.3 Funcionamiento General

El funcionamiento del sistema se basa en una arquitectura cliente-servidor, donde las solicitudes realizadas por los usuarios son enviadas mediante peticiones HTTP hacia los servicios que componen la aplicación.

Antes de acceder a las funcionalidades disponibles, el usuario debe autenticarse utilizando sus credenciales. Una vez validada su identidad, el sistema genera un token de autenticación que será utilizado para autorizar las solicitudes posteriores a los distintos servicios disponibles.

Cada operación realizada por el usuario permite consultar, registrar, modificar o eliminar información correspondiente a los distintos procesos administrados por la plataforma. La información es procesada por la lógica de negocio de cada servicio y posteriormente almacenada en la base de datos, garantizando la integridad y consistencia de los datos.

Gracias a esta organización, el sistema permite gestionar de forma centralizada el inventario de equipos, las especificaciones técnicas, el historial de mantenimientos, las canciones del repertorio, las giras, los eventos, la logística de transporte, los productos de merchandising y las transacciones financieras, proporcionando una visión integral del funcionamiento de una banda musical.

3.4 Módulos del Sistema

El proyecto Instrumentum está compuesto por diversos módulos funcionales, cada uno orientado a resolver una necesidad específica dentro de la administración de una banda musical. En conjunto, estos módulos permiten mantener organizada la información y facilitar la gestión de los diferentes procesos que forman parte de la actividad artística y técnica.

Los principales módulos que conforman el sistema son los siguientes:

- ° **Autenticación:** encargado de controlar el acceso al sistema mediante la validación de usuarios y la generación de tokens de autenticación.
- ° **Usuarios:** permite administrar la información de bandas como también los integrantes, considerando tanto músicos como personal técnico.
- ° **Inventario:** centraliza el registro y administración de todos los equipos e instrumentos utilizados por la agrupación.
- ° **Especificaciones Técnicas:** almacena la información técnica asociada a los equipos, diferenciando entre instrumentos musicales y dispositivos electrónicos.
- ° **Mantenimiento:** registra el historial de servicios realizados sobre los equipos y facilita el seguimiento de las intervenciones técnicas efectuadas.
- ° **Eventos:** administra las presentaciones realizadas por la banda, permitiendo mantener un registro organizado de sus actividades.
- ° **Giras:** permite planificar y administrar las giras, incluyendo las distintas paradas, información de alojamiento y transporte.
- ° **Logística:** organiza el transporte del equipamiento mediante contenedores y su respectiva distribución durante los desplazamientos.
- ° **Merchandising:** administra los productos comercializados por la banda y registra las ventas realizadas durante eventos u otras actividades.
- ° **Finanzas:** mantiene un registro de las transacciones económicas asociadas a la banda, facilitando el control de ingresos y egresos.
- ° **API Gateway:** actúa como punto de entrada para las solicitudes realizadas por los clientes, centralizando el acceso a los servicios disponibles y facilitando la administración de las peticiones dentro de la arquitectura.

En las siguientes secciones del presente informe se describirá con mayor detalle la arquitectura del sistema y la función que cumple cada uno de estos componentes dentro de la solución implementada.

4. Arquitectura del Sistema

4.1 Arquitectura General

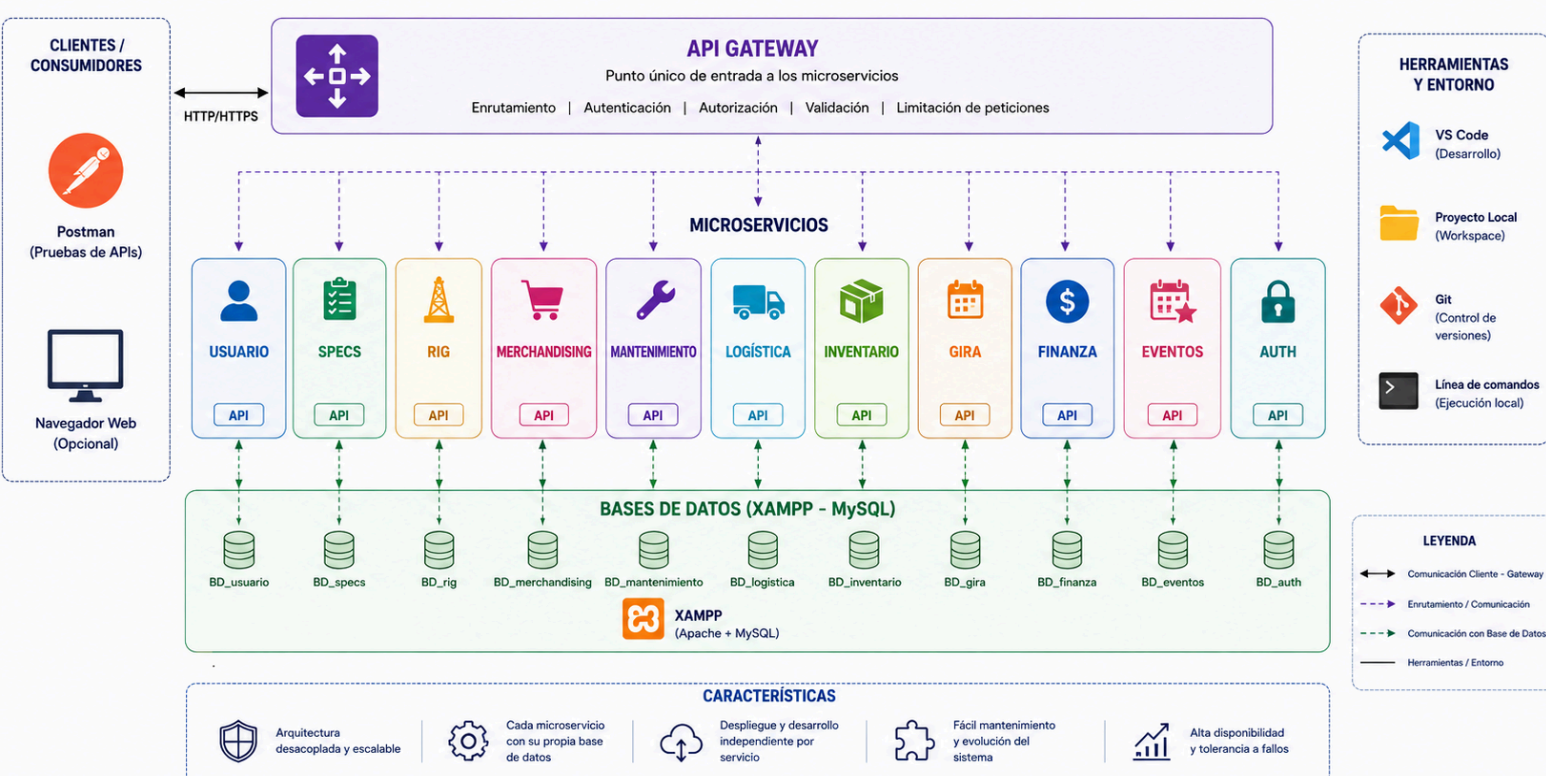
Instrumentum fue desarrollado utilizando una **arquitectura basada en microservicios**, en la cual cada componente del sistema es responsable de un conjunto específico de funcionalidades y opera de manera independiente. Este enfoque permite separar las responsabilidades de cada módulo, facilitando el mantenimiento, la escalabilidad y la incorporación de nuevas funcionalidades sin afectar el funcionamiento del resto de la aplicación.

A diferencia de una arquitectura monolítica, los microservicios permiten que cada servicio pueda evolucionar de forma independiente, manteniendo una comunicación controlada a través de protocolos HTTP y centralizando el acceso mediante un API Gateway.

En la arquitectura implementada, cada microservicio posee su propia lógica de negocio organizada mediante una arquitectura por capas (Controller, Service y Repository), además de administrar su persistencia de datos utilizando Spring Data JPA y una base de datos MySQL.

Por otra parte, el acceso a los servicios se encuentra protegido mediante autenticación basada en tokens JWT, garantizando que únicamente los usuarios autorizados puedan consumir los recursos expuestos por la aplicación.

ARQUITECTURA DE MICROSERVICIOS – PLATAFORMA ALFA



La arquitectura está compuesta por un conjunto de componentes especializados que trabajan de forma coordinada para entregar las distintas funcionalidades del sistema. A continuación, se describe el propósito de cada uno de ellos.

4.1.1 API Gateway

El API Gateway constituye el punto único de entrada para todas las solicitudes realizadas por los clientes hacia la plataforma Instrumentum. Su función principal consiste en recibir las peticiones HTTP provenientes de aplicaciones cliente y redirigirlas al microservicio correspondiente según la ruta solicitada.

Esta centralización permite ocultar la estructura interna de la arquitectura, evitando que los clientes deban conocer la ubicación o dirección específica de cada microservicio.

Además de realizar el enrutamiento de las solicitudes, el Gateway facilita la aplicación de políticas comunes para toda la plataforma, como el control de acceso mediante tokens de autenticación, el manejo uniforme de las peticiones y la simplificación de la comunicación entre el cliente y los distintos servicios del sistema.

Desde el punto de vista arquitectónico, el API Gateway actúa como intermediario entre el cliente y los microservicios, mejorando la organización del sistema y permitiendo que futuras modificaciones en la infraestructura no afecten directamente a las aplicaciones consumidoras.

4.1.3 Microservicios

La plataforma Instrumentum se encuentra dividida en diversos microservicios, cada uno orientado a administrar un dominio específico del sistema. Esta organización permite separar las responsabilidades funcionales y mantener un bajo nivel de acoplamiento entre los distintos componentes de la aplicación.

A continuación, se presenta una descripción general de los microservicios que conforman la solución implementada.

Auth Service

Microservicio encargado de la autenticación de usuarios. Gestiona la validación de credenciales y la generación de tokens JWT utilizados para autorizar el acceso a los recursos protegidos del sistema.

Usuario Service

Administra la información de los usuarios registrados, incluyendo músicos y personal técnico, permitiendo realizar las operaciones de creación, consulta, actualización y eliminación de registros.

Inventario Service

Gestiona el inventario de equipos e instrumentos pertenecientes a la banda, manteniendo la información relacionada con marcas, categorías, propietarios y demás datos generales del equipamiento.

Specs Service

Administra las especificaciones técnicas de los equipos registrados, diferenciando la información correspondiente a instrumentos musicales y equipos electrónicos.

Mantenimiento Service

Registra el historial de mantenimientos efectuados sobre cada equipo, almacenando información referente a las intervenciones técnicas realizadas y facilitando el seguimiento del estado del equipamiento.

Evento Service

Gestiona la información de los eventos y presentaciones organizadas por la banda, permitiendo mantener un registro estructurado de sus actividades.

Gira Service

Administra las giras realizadas por la agrupación, incluyendo las distintas paradas planificadas y la información logística asociada a cada una de ellas.

Logística Service

Permite gestionar la organización del transporte del equipamiento mediante contenedores, facilitando el control de los equipos movilizados durante giras y eventos.

Merchandising Service

Administra el catálogo de productos comercializados por la banda y registra las ventas efectuadas durante eventos u otras actividades.

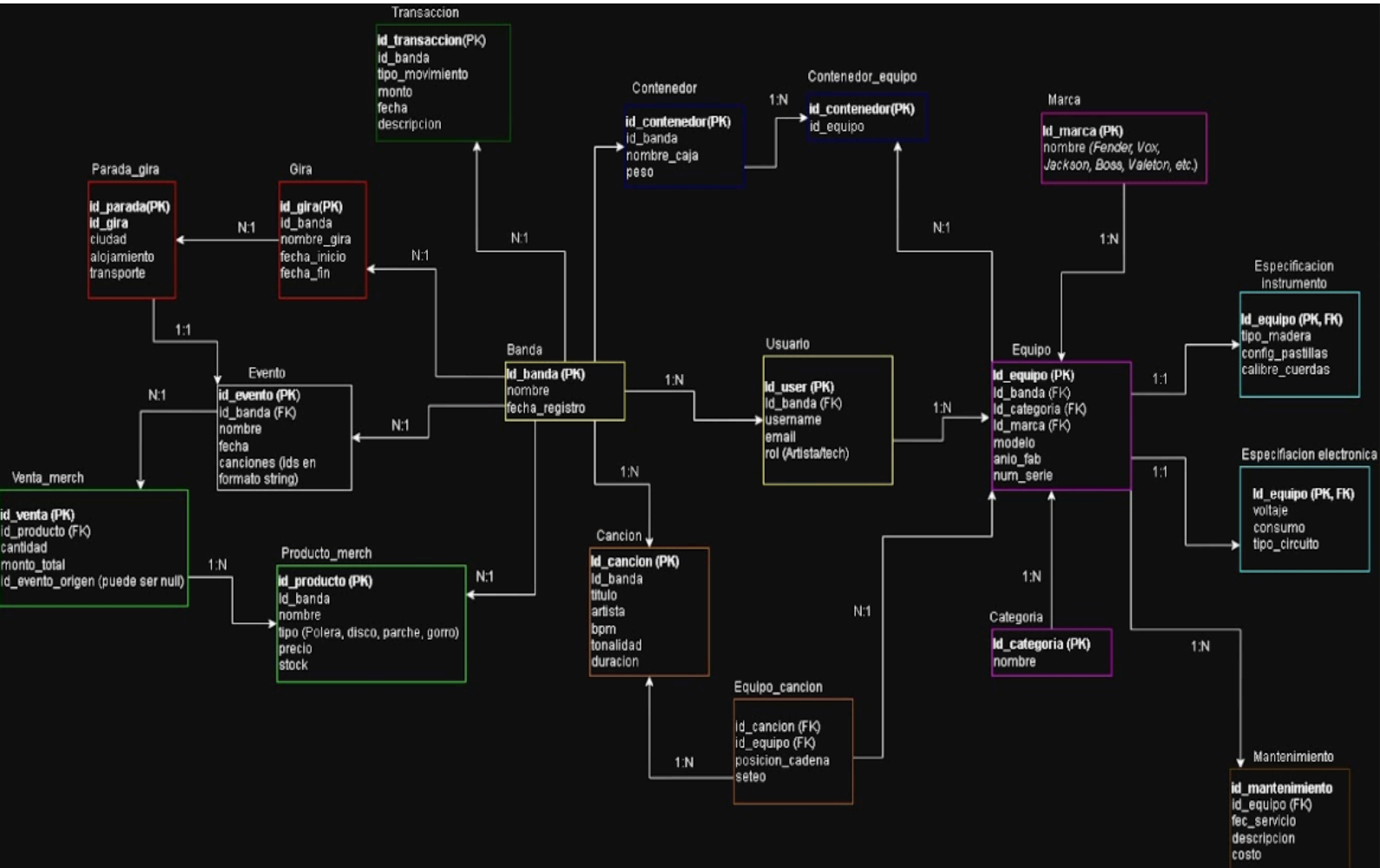
Finanza Service

Gestiona el registro de las transacciones económicas de la banda, permitiendo mantener un control de ingresos, egresos y demás movimientos financieros asociados a la actividad musical.

Documentación de servicios

Cada microservicio expone su propia documentación OpenAPI mediante SpringDoc. Posteriormente, el API Gateway centraliza dichas definiciones para ofrecer una única interfaz Swagger UI desde la cual es posible consultar y probar los endpoints disponibles sin necesidad de acceder individualmente a cada servicio.

5. Modelo de Base de Datos



5.1 Banda

La entidad Banda constituye el núcleo del sistema, ya que representa a cada agrupación musical registrada. Almacena el identificador de la banda (`id_banda`), su nombre y la `fecha_registro`. Desde esta entidad se relacionan gran parte de los módulos del sistema, permitiendo asociar usuarios, canciones, eventos, giras, productos de merchandising, contenedores y transacciones a una agrupación específica. **Relaciones:**

- Una banda puede tener múltiples usuarios (1:N).
- Una banda puede registrar varias canciones (1:N).

- Una banda puede organizar múltiples eventos (1:N).
- Una banda puede realizar varias giras (1:N).
- Una banda puede disponer de diferentes productos de merchandising (1:N).
- Una banda puede poseer varios contenedores de transporte (1:N).
- Una banda puede registrar múltiples transacciones financieras (1:N).

5.2 Usuario

La entidad Usuario representa a los integrantes del sistema, ya sean artistas o técnicos. Cada usuario pertenece a una única banda (id_banda) y almacena información básica como el identificador (id_user), username, email y rol (Artista/tech). Esta estructura permite administrar los miembros de cada agrupación musical.

Relaciones:

- Una banda posee muchos usuarios (N:1 hacia Banda).
- Un usuario puede estar relacionado con varios equipos (1:N hacia Equipo).

5.3 Equipo

La entidad Equipo corresponde al inventario principal del sistema. Aquí se almacenan todos los instrumentos y equipos electrónicos utilizados, incluyendo información y llaves foráneas como:

- modelo
- anio_fab (año de fabricación)
- num_serie (número de serie)
- id_marca (FK)
- id_categoria (FK)
- id_banda (FK)

Esta entidad es una de las más importantes del sistema, ya que se relaciona con mantenimiento, especificaciones técnicas y configuración de canciones. **Relaciones:**

- Se relaciona con un usuario específico (N:1 hacia Usuario).
- Pertenece a una banda (N:1 hacia Banda).
- Una marca puede pertenecer a muchos equipos (N:1 hacia Marca).
- Una categoría clasifica múltiples equipos (N:1 hacia Categoría).
- Posee una única especificación técnica dependiendo de su tipo (1:1 hacia Especificacion_instrumento o Especificacion_electronica).
- Un equipo puede registrar múltiples mantenimientos (1:N hacia Mantenimiento).
- Un equipo puede utilizarse en varias canciones a través de una tabla intermedia (1:N hacia Equipo_cancion).
- Un equipo puede estar en varios contenedores a través de una tabla intermedia (1:N hacia Contenedor_equipo).

5.4 Marca

La entidad Marca almacena los fabricantes del equipamiento utilizado por las bandas (identificador `id_marca` y nombre), como Fender, Vox, Jackson, Boss, Valeton, etc. Separar esta información en una entidad independiente evita la duplicación de datos y facilita futuras consultas.

Relaciones:

- Una marca puede estar asociada a múltiples equipos (1:N hacia Equipo).

5.5 Categoría

La entidad Categoría clasifica el equipamiento según su función (identificador `id_categoria` y nombre). Ejemplos: Guitarra, Bajo, Pedal, Amplificador, Micrófono. Esto permite organizar el inventario y realizar búsquedas o filtros por tipo de equipo.

Relaciones:

- Una categoría puede contener muchos equipos (1:N hacia Equipo).

5.6 Especificación Instrumento

Esta entidad almacena las características técnicas exclusivas de los instrumentos musicales. Entre sus atributos se encuentran el `tipo_madera`, `config_pastillas` y `calibre_cuerdas`.

Relaciones:

- Mantiene una relación de uno a uno (1:1) con Equipo, compartiendo la llave principal (`id_equipo` actúa como PK y FK), ya que únicamente los instrumentos requieren este tipo de información.

5.7 Especificación Electrónica

Esta entidad almacena las características técnicas de los equipos electrónicos. Registra información como el voltaje, consumo y `tipo_circuito`.

Relaciones:

- Al igual que la especificación de instrumentos, mantiene una relación de uno a uno (1:1) con la entidad Equipo compartiendo el `id_equipo` (PK, FK). Esta separación permite especializar la información técnica según el tipo de equipamiento.

5.8 Mantenimiento

La entidad Mantenimiento registra el historial de servicios realizados sobre cada equipo. Cada registro almacena el identificador (id_mantenimiento), fec_servicio, descripcion y costo. Esta información permite llevar un control del estado del equipamiento y mantener un historial de intervenciones técnicas.

Relaciones:

- Un equipo puede tener múltiples mantenimientos (N:1 hacia Equipo mediante id_equipo).

5.9 Canción

La entidad Cancion almacena el repertorio musical de cada banda. Cada canción registra información como id_cancion, titulo, artista, bpm, tonalidad y duracion. Esta entidad permite construir configuraciones específicas de equipamiento para cada interpretación.

Relaciones:

- Pertenece a una banda específica (N:1 hacia Banda mediante id_banda).
- Una canción puede utilizar múltiples equipos (1:N hacia Equipo_cancion).

5.10 Equipo_Canción

Esta entidad actúa como tabla intermedia entre Canción y Equipo, resolviendo la relación de muchos a muchos. Además de relacionar ambas entidades (id_cancion e id_equipo), almacena información propia de la configuración utilizada durante la interpretación de una canción:

- posicion_cadena (posición dentro de la cadena de señal).
- seteo (configuración del equipo).

Gracias a esta estructura es posible definir un *rig* diferente para cada canción del repertorio.

5.11 Evento

La entidad Evento representa las presentaciones o actividades realizadas por una banda. Cada evento almacena información como el id_evento, nombre, fecha y las canciones (almacenadas como IDs en formato string). Esta información permite relacionar posteriormente ventas y logística durante cada presentación.

Relaciones:

- Pertenece a una banda (N:1 hacia Banda mediante id_banda).
- Un evento se asocia de forma exacta a una parada de una gira (1:1 con Parada_gira).
- Un evento puede generar varias ventas de merchandising (1:N hacia Venta_merch).

5.12 Gira

La entidad Gira permite registrar los recorridos realizados por una banda. Almacena información como el id_gira, nombre_gira, fecha_inicio y fecha_fin. Relaciones:

- Pertenece a una banda (N:1 hacia Banda mediante id_banda).
- Cada gira puede estar compuesta por varias paradas (1:N hacia Parada_gira).

5.13 Parada de Gira

La entidad Parada_gira representa cada destino visitado durante un tour. Se almacena información como id_parada, ciudad, alojamiento y transporte.

Relaciones:

- Pertenece a una gira en particular (N:1 hacia Gira mediante id_gira).
- Se relaciona de manera uno a uno (1:1) con la entidad Evento, vinculando la logística del viaje directamente con el show agendado.

5.14 Producto Merchandising

La entidad Producto_merch administra los productos comercializados por la banda (poleras, discos, parches, gorros, etc.). Cada producto almacena su id_producto, nombre, tipo, precio y stock disponible.

Relaciones:

- Pertenece a una banda (N:1 hacia Banda mediante id_banda).
- Un producto puede estar en múltiples ventas (1:N hacia Venta_merch).

5.15 Venta Merchandising

La entidad Venta_merch registra las transacciones de los productos de mercancía. Cada venta almacena el id_venta, la cantidad, el monto_total y el id_evento_origen (que puede ser nulo si la venta no se realizó en un show).

Relaciones:

- Registra la venta de un producto en particular (N:1 hacia Producto_merch mediante id_producto).
- Puede asociarse a un evento específico (N:1 hacia Evento mediante id_evento_origen).

5.16 Contenedor

La entidad Contenedor representa las cajas o flightcases utilizados para transportar equipamiento. Cada contenedor almacena el id_contenedor, nombre_caja y peso.

Relaciones:

- Pertenece a una banda (N:1 hacia Banda mediante id_banda).
- Un contenedor puede transportar varios equipos (1:N hacia Contenedor_equipo).

5.17 Contenedor_Equipo

Corresponde a la tabla intermedia que relaciona los equipos con los contenedores. Resuelve la relación muchos a muchos utilizando el id_contenedor y el id_equipo. Gracias a esta relación es posible conocer qué equipos específicos viajan dentro de cada caja de transporte.

5.18 Transacción

La entidad Transaccion registra los movimientos financieros asociados a una banda. Entre sus atributos se encuentran el id_transaccion, tipo_movimiento, monto, fecha y descripcion. Esta información permite llevar un control básico de ingresos y egresos relacionados con la actividad musical.

Relaciones:

- Pertenece a una banda (N:1 hacia Banda mediante id_banda).

Resumen del modelo

El Modelo Entidad–Relación de Instrumentum integra los distintos procesos involucrados en la gestión de una banda musical dentro de una única base de datos relacional. Su diseño permite administrar usuarios, inventario, mantenimiento, repertorio, giras, eventos, logística, merchandising y finanzas mediante entidades especializadas y relaciones claramente definidas. Esta organización favorece la integridad referencial, evita la duplicidad de datos y facilita la escalabilidad del sistema, permitiendo incorporar nuevos módulos o funcionalidades sin afectar la estructura general de la base de datos

6. Servicio de Autenticación

El servicio de autenticación de Instrumentum utiliza Spring Security junto con JWT para gestionar el acceso seguro a los recursos del sistema.

Este microservicio permite registrar usuarios, validar credenciales y generar tokens de autenticación que serán utilizados para identificar usuarios en futuras solicitudes.

La configuración del servicio se encuentra definida mediante clases separadas por responsabilidad:

- SecurityConfig: configuración de seguridad y protección de endpoints.
- AuthController: exposición de endpoints REST para registro e inicio de sesión.
- AuthService: lógica de autenticación y validación de usuarios.
- JwtService: generación de tokens JWT.
- UsuarioRepository: acceso a los datos almacenados en MySQL.

La persistencia de usuarios se realiza mediante Spring Data JPA utilizando una base de datos MySQL.

¿En qué consiste?

El servicio de autenticación tiene como objetivo gestionar el acceso seguro de los usuarios al sistema, permitiendo validar sus credenciales y controlar el acceso a los recursos protegidos mediante un mecanismo basado en JSON Web Token (JWT).

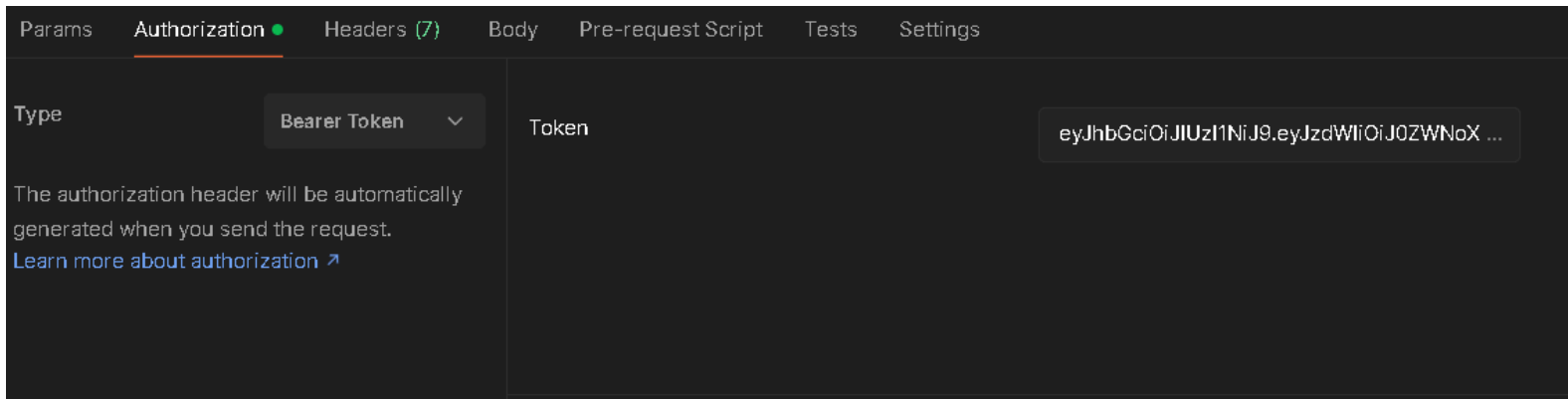
Este microservicio se encarga de recibir las solicitudes de inicio de sesión, verificar la información entregada por el usuario contra los datos almacenados, generar un token de seguridad cuando la autenticación es correcta y permitir que dicho token sea utilizado posteriormente para acceder a los endpoints protegidos del sistema.

La autenticación implementada utiliza un esquema stateless, donde el servidor no mantiene sesiones de usuario activas. En su lugar, cada petición autenticada debe incluir el token generado previamente, permitiendo validar la identidad del usuario en cada solicitud.

6.1 Implementación mediante Token

La autenticación se implementa utilizando **JWT (JSON Web Token)** como mecanismo de autorización.

Cuando un usuario inicia sesión correctamente, el servicio genera un token firmado que contiene información necesaria para identificar al usuario autenticado. Este token es enviado al cliente, quien debe almacenarlo y enviarlo en las siguientes solicitudes mediante el encabezado HTTP:



El sistema valida el token antes de permitir el acceso a los recursos protegidos, verificando que:

- El token exista.
- La firma sea válida.
- La información del usuario sea correcta.

Si la validación es exitosa, la petición continúa hacia el controlador correspondiente. En caso contrario, el acceso es rechazado.

6.2 Flujo de autenticación

El funcionamiento del proceso es el siguiente:



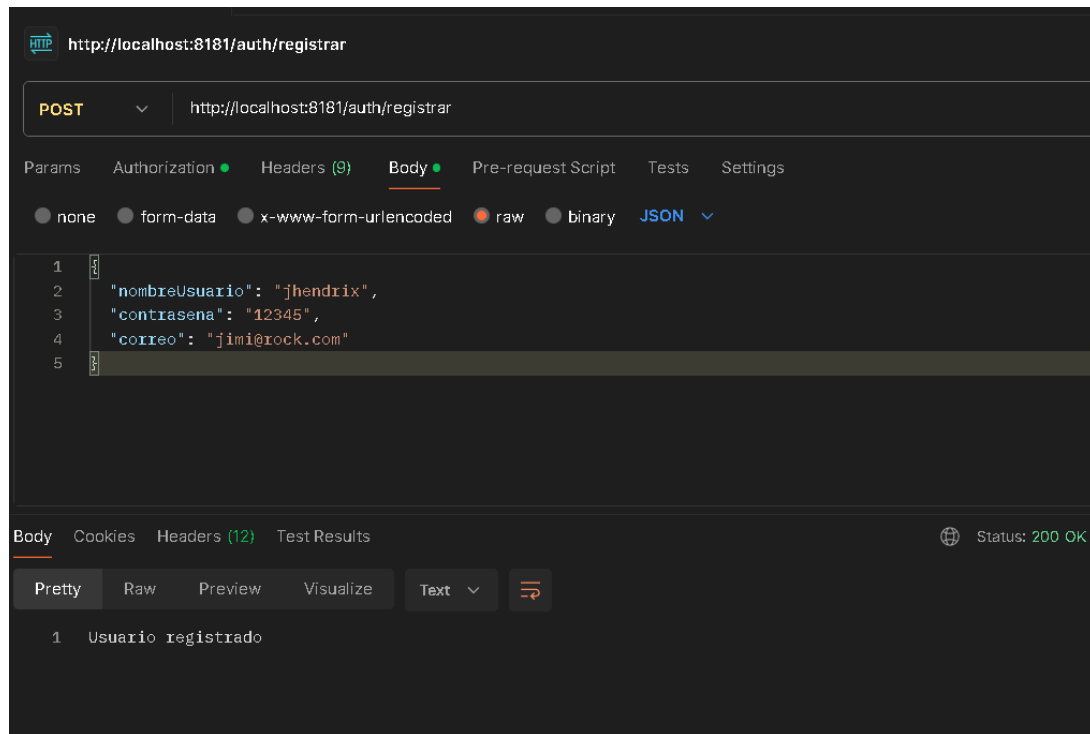
Validación del token



Acceso al recurso

1. Usuario

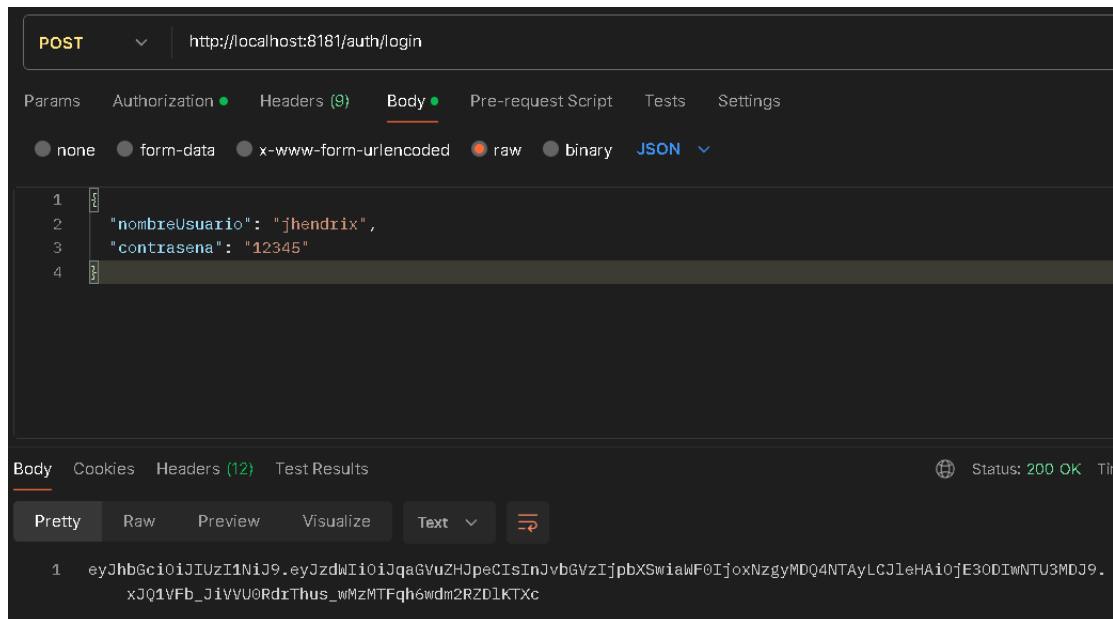
El usuario ingresa sus credenciales mediante la aplicación cliente, enviando datos como nombre de usuario/correo y contraseña al endpoint de autenticación.



2. Login

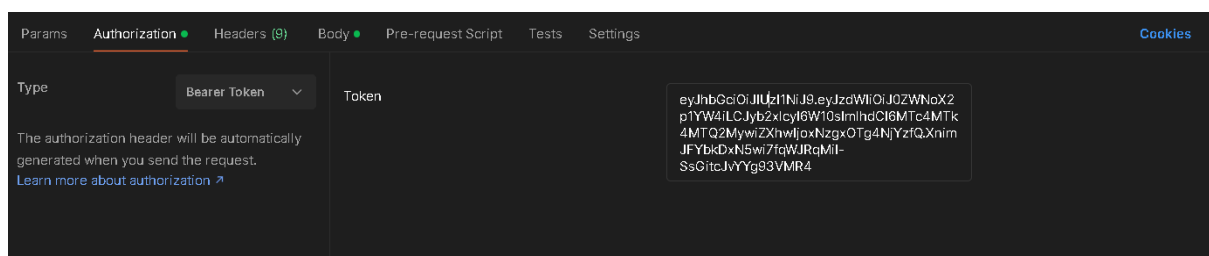
El cliente realiza una solicitud HTTP al controlador encargado de autenticación.

El controlador recibe la información enviada y la deriva al servicio correspondiente para realizar el proceso de validación.



3. Validación de credenciales

El servicio de autenticación consulta la información del usuario almacenada mediante el repositorio correspondiente. Se comparan las credenciales entregadas con los datos registrados en el sistema utilizando un encriptador (BCryptPasswordEncoder) para garantizar la seguridad y evitar que la contraseña se maneje en texto plano. Si los datos son correctos, el usuario es considerado válido y continúa el flujo de autenticación.

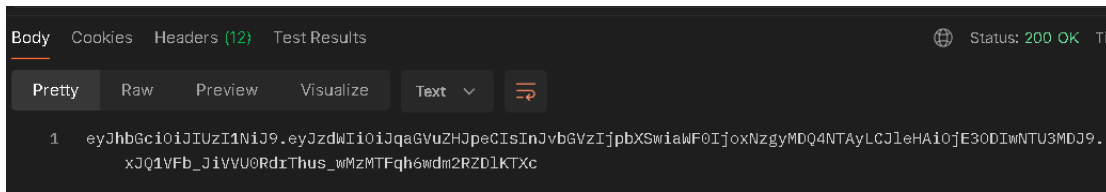


4. Generación del JWT

Una vez autenticado el usuario, el sistema genera un token JWT.

Este token contiene información del usuario autenticado y posee una fecha de expiración definida.

El token es firmado utilizando la configuración de seguridad del sistema para evitar modificaciones no autorizadas.



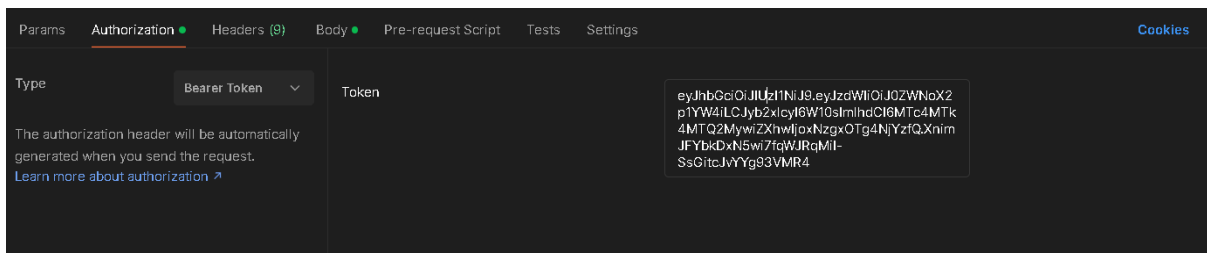
5. Cliente almacena el token

El cliente recibe el JWT como respuesta del servicio de autenticación.

Posteriormente debe almacenarlo y utilizarlo para las solicitudes futuras que requieran autenticación.

6. Peticiones autenticadas

Cuando el usuario intenta acceder a un recurso protegido, el cliente incluye el token en el encabezado de la solicitud HTTP:



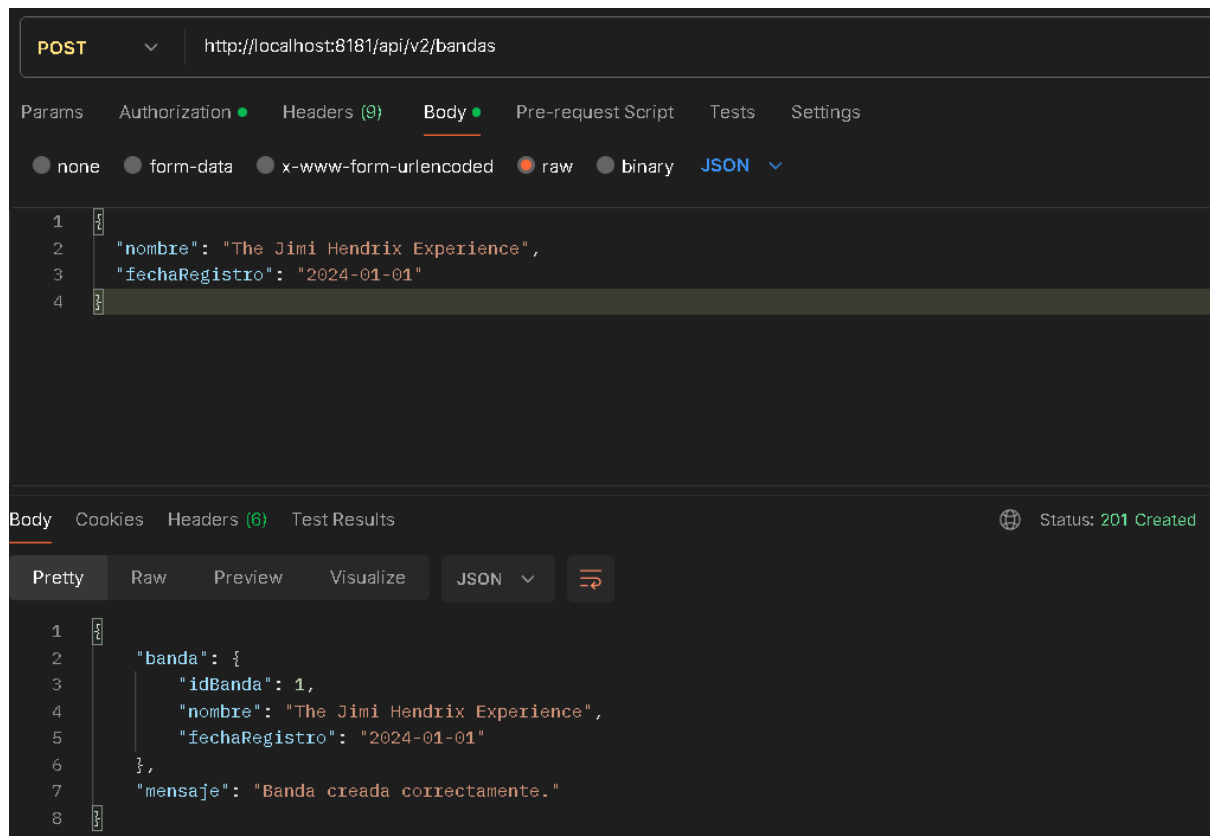
7. Validación del token

Antes de permitir el acceso, el filtro de seguridad intercepta la solicitud y analiza el token recibido.

Se valida:

- Existencia del token.
- Integridad de la firma.
- Vigencia del token.
- Información asociada al usuario.

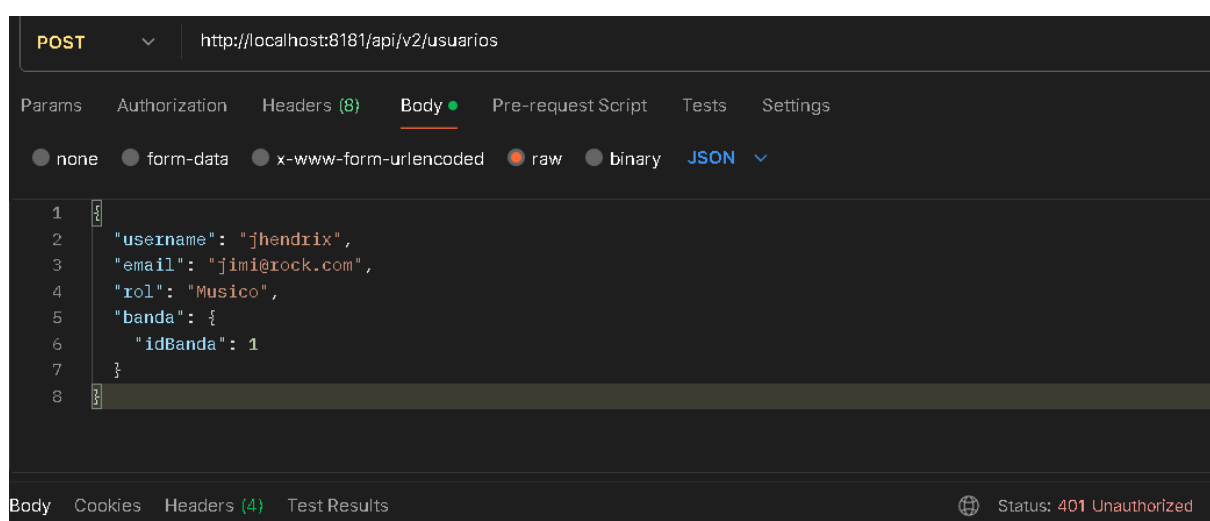
Si el token es válido, se crea el contexto de seguridad correspondiente.



8. Acceso al recurso

Luego de validar correctamente el token, la solicitud continúa hacia el controlador solicitado permitiendo ejecutar la operación correspondiente.

Si el token no es válido, el sistema devuelve una respuesta indicando que la solicitud no está autorizada.



6.3 Configuración del servicio de autenticación.

Seguridad y JWT

Estas dependencias permiten implementar la protección de endpoints, cifrado de contraseñas y generación de tokens de autenticación.

Spring Security: (spring-boot-starter-security)

Proporciona la infraestructura de seguridad utilizada por la aplicación.

Sus funcionalidades principales son:

- Configuración de rutas protegidas.
- Manejo del contexto de seguridad.
- Protección de endpoints REST.
- Integración con mecanismos de autenticación.
- Codificación segura de contraseñas mediante BCrypt.

En el proyecto es utilizada principalmente en la clase:

- SecurityConfig

donde se define la cadena de seguridad del servicio.

JSON Web Token (JWT)

Las dependencias utilizadas son:

- jjwt-api
- jjwt-impl
- jjwt-jackson

Estas librerías permiten trabajar con tokens JWT dentro de la aplicación.

Sus funciones son:

- jjwt-api

Proporciona las interfaces necesarias para crear y manipular tokens JWT.

Es utilizada por:

- JwtService

para construir el token.

jjwt-impl:

Contiene la implementación interna encargada de:

- Firmar tokens.
- Generar la estructura JWT.
- Aplicar algoritmos criptográficos.

En este proyecto se utiliza el algoritmo:

- SignatureAlgorithm.HS256

jjwt-jackson

Permite convertir la información almacenada dentro del JWT a formato JSON.

En el proyecto se utiliza para almacenar información adicional dentro del token, como los roles del usuario:

- .claim("roles", roles)

Dependencias Web y Estructura

Spring Boot Web: (spring-boot-starter-web)

Permite crear la API REST del microservicio.

Proporciona:

- Servidor Tomcat embebido.
- Creación de controladores REST.
- Manejo de solicitudes HTTP.

En el proyecto permite implementar:

- @RestController
- @PostMapping
- @RequestMapping

utilizado en:

- AuthController

Spring Boot Validation: (spring-boot-starter-validation)

Permite validar los datos recibidos desde los clientes.

La dependencia deja preparada la estructura para aplicar validaciones sobre DTOs como:

- AuthRequest

Dependencias de Base de Datos

Spring Data JPA: (spring-boot-starter-data-jpa)

Permite realizar la conexión entre la aplicación y la base de datos mediante entidades y repositorios.

Sus funciones son:

- Mapear clases Java como tablas.
- Ejecutar consultas.
- Administrar persistencia de datos.

En el proyecto se utiliza mediante:

- UsuarioRepository

MySQL Connector (mysql-connector-j)

Es el controlador que permite la comunicación entre Spring Boot y MySQL.

La conexión está configurada mediante:

- spring.datasource.url=jdbc:mysql://localhost:3306/db_auth

La base de datos utilizada corresponde a:

- db_auth

Herramientas adicionales y documentación

Lombok: Permite reducir código repetitivo generando automáticamente:

- Getters.

- Setters.
- Constructores.
- Métodos auxiliares.

Es utilizado en las entidades:

- Usuario
- Rol
- AuthRequest

mediante anotaciones como:

- @Data
- @NoArgsConstructor
- @AllArgsConstructor

Swagger / OpenAPI: (springdoc-openapi-starter-webmvc-ui)

Permite generar automáticamente la documentación de la API, en el proyecto está configurado mediante:

- springdoc.api-docs.path=/auth/api-docs
- springdoc.swagger-ui.path=/swagger-ui.html

Permitiendo probar endpoints como:

- POST /auth/login
- POST /auth/registrarse

Testing

Dependencias:

- spring-boot-starter-test
- spring-security-test

Permiten realizar pruebas unitarias y pruebas relacionadas con seguridad.

6.3.2 Clases principales del servicio de autenticación

SecurityConfig

La clase SecurityConfig contiene la configuración principal de seguridad del microservicio.

Sus responsabilidades son:

- Configurar las rutas protegidas y públicas.
- Definir el mecanismo de autenticación.
- Registrar los filtros de seguridad.
- Deshabilitar configuraciones innecesarias para trabajar con JWT.
- Permitir el acceso mediante tokens en lugar de sesiones tradicionales.

Esta configuración permite que el sistema utilice autenticación basada en tokens.

```
package cl.instrumentum.service_auth.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http.csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll()
                .anyRequest().authenticated()
            ).build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

La configuración establece que “.requestMatchers("/auth/**").permitAll()”, permite acceder sin autenticación a:

- POST /auth/login
- POST /auth/registrar

Mientras que “.anyRequest().authenticated()” obliga a que cualquier otro recurso requiera autenticación.

Además, registra “BCryptPasswordEncoder” para realizar el cifrado seguro de contraseñas.

AuthService

La clase `cl.instrumentum.service_auth.service.AuthService` contiene la lógica principal del proceso de autenticación.

Sus funciones son:

- Registrar usuarios.
- Encriptar contraseñas.
- Buscar usuarios en la base de datos.
- Validar credenciales.
- Obtener roles del usuario.
- Solicitar la generación del JWT.

```
import cl.instrumentum.service_auth.model.Usuario;
import org.springframework.stereotype.Service;

@Service
public class AuthService {

    @Autowired
    private UsuarioRepository usuarioRepo;

    @Autowired
    private JwtService jwtService;

    @Autowired
    private PasswordEncoder passwordEncoder;

    public String registrar(Usuario usuario) {
        usuario.setContrasena(passwordEncoder.encode(usuario.getContrasena()));
        usuarioRepo.save(usuario);
        return "Usuario registrado";
    }

    public String login(String username, String password){
        Usuario user = usuarioRepo.findByNombreUsuario(username)
            .orElseThrow(() -> new RuntimeException(message: "Usuario no encontrado"));

        if(passwordEncoder.matches(password, user.getContrasena())) {
            List<String> roles = user.getRoles().stream()
                .map(Rol::getNombreRol).collect(Collectors.toList());
            return jwtService.generarToken(username, roles);
        }
        throw new RuntimeException(message: "Credenciales inválidas");
    }
}
```

Durante el registro `"passwordEncoder.encode(usuario.getContrasena())"`, convierte la contraseña en un hash seguro antes de almacenarla.

Durante el login “passwordEncoder.matches()”, compara la contraseña ingresada con la contraseña almacenada. Si la validación es correcta, genera el token mediante jwtService.generarToken()

JwtService

La clase “cl.instrumentum.service_auth.service.JwtService” es responsable de crear los tokens JWT.

```
1  package cl.instrumentum.service_auth.service;
2
3  import java.util.Date;
4  import java.util.List;
5  import org.springframework.beans.factory.annotation.Value;
6  import org.springframework.stereotype.Service;
7  import io.jsonwebtoken.Jwts;
8  import io.jsonwebtoken.SignatureAlgorithm;
9  import io.jsonwebtoken.security.Keys;
10
11  @Service
12  public class JwtService {
13
14      @Value("${jwt.secret}")
15      private String secreto;
16
17      public String generarToken(String username, List<String> roles){
18          long dosHorasEnMilisegundos = 1000*60*60*2;
19
20          return Jwts.builder()
21              .setSubject(username)
22              .claim("roles", roles)
23              .setIssuedAt(new Date())
24              .setExpiration(new Date(System.currentTimeMillis() + dosHorasEnMilisegundos))
25              .signWith(Keys.hmacShaKeyFor(secreto.getBytes()), SignatureAlgorithm.HS256)
26              .compact();
27      }
28  }
```

El método “generarToken()” realiza:

- Asignación del usuario como sujeto del token.
- Incorporación de roles.
- Definición de fecha de creación.
- Definición de expiración.
- Firma digital del token.

El token tiene una duración configurada de 2 horas y utiliza HS256 como algoritmo de firma.

AuthController

La clase "cl.instrumentum.service_auth.controller.AuthController" expone los endpoints REST del servicio.

```
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import cl.instrumentum.service_auth.dto.AuthRequest;
import cl.instrumentum.service_auth.model.Usuario;
import cl.instrumentum.service_auth.service.AuthService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;

@RestController
@RequestMapping("/auth")
@Tag(name = "Autenticación", description = "Endpoints para registro y login de usuarios")
public class AuthController {

    @Autowired
    private AuthService authService;

    @Operation(summary = "Registrar un nuevo usuario", description = "Guarda el usuario con la contraseña encriptada")
    @PostMapping("/registrar")
    public ResponseEntity<String> registrar(@RequestBody Usuario usuario) {
        return ResponseEntity.ok(authService.registrar(usuario));
    }

    @Operation(summary = "Iniciar sesión", description = "Retorna un Token JWT si las credenciales son válidas")
    @PostMapping("/login")
    public ResponseEntity<String> login(@RequestBody AuthRequest request){
        try{
            String token = authService.login(request.getNombreUsuario(), request.getContraseña());
            return ResponseEntity.ok(token);
        } catch (RuntimeException e) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(e.getMessage());
        }
    }
}
```

Endpoints disponibles:

- Registro:
 - POST /auth/registrar

Permite crear un nuevo usuario.

- Login:
 - POST /auth/login

Permite autenticar un usuario y obtener un JWT. Si las credenciales son correctas retorna un Token JWT, si son incorrectas, 401 Unauthorized

UsuarioRepository

La interfaz `cl.instrumentum.service_auth.repository.UsuarioRepository`, permite acceder a los usuarios almacenados en MySQL.

```
package cl.instrumentum.service_auth.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import cl.instrumentum.service_auth.model.Usuario;
import java.util.Optional;

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {

    Optional<Usuario> findByNombreUsuario(String nombreUsuario);
}
```

Extiende `JpaRepository<Usuario, Long>` y agrega la búsqueda `findByNombreUsuario()` utilizada durante el inicio de sesión.

Modelos del servicio de autenticación

El microservicio de autenticación utiliza modelos JPA para representar las entidades persistidas en la base de datos MySQL.

Las entidades principales corresponden a:

- Usuario
- Rol

Estas clases permiten mapear los objetos Java con las tablas correspondientes dentro de la base de datos.

Entidad Usuario

La clase `cl.instrumentum.service_auth.model.Usuario` representa la información de los usuarios registrados en el sistema. Esta entidad se encuentra asociada a la tabla “usuarios”.

Sus atributos principales son:

- `private Long id;`
- `private String nombreUsuario;`

- private String contrasena;
- private String correo;

Sus responsabilidades son:

- Almacenar la información del usuario.
- Mantener la contraseña cifrada.
- Identificar al usuario dentro del sistema.
- Relacionar los permisos mediante sus roles.

La contraseña no se almacena directamente, sino que es cifrada mediante BCryptPasswordEncoder antes de ser guardada en la base de datos.

```

1  package cl.instrumentum.service_auth.model;
2
3  import jakarta.persistence.*;
4  import lombok.Data;
5  import lombok.NoArgsConstructor;
6  import lombok.AllArgsConstructor;
7  import java.util.HashSet;
8  import java.util.Set;
9
10 @Entity
11 @Table(name = "usuarios")
12 @Data
13 @NoArgsConstructor
14 @AllArgsConstructor
15 public class Usuario {
16
17     @Id
18     @GeneratedValue(strategy = GenerationType.IDENTITY)
19     private Long id;
20
21     @Column(unique = true)
22     private String nombreUsuario;
23     private String contrasena;
24     private String correo;
25
26     @ManyToMany(fetch = FetchType.EAGER)
27     @JoinTable(
28         name = "usuario_roles",
29         joinColumns = @JoinColumn(name = "usuario_id"),
30         inverseJoinColumns = @JoinColumn(name = "rol_id")
31     )
32     private Set<Rol> roles = new HashSet<>();
33 }

```

Relación Usuario - Rol

La entidad Usuario mantiene una relación muchos a muchos con la entidad "cl.instrumentum.service_auth.model.Rol" mediante @ManyToMany(fetch = FetchType.EAGER) Esta relación permite que:

- Un usuario tenga múltiples roles.
- Un rol pueda pertenecer a múltiples usuarios.

La tabla intermedia generada corresponde a usuario_roles

Esta estructura permite extender posteriormente el sistema de autorización basado en permisos.

Entidad Rol

La clase `cl.instrumentum.service_auth.model.Rol` representa los roles disponibles dentro del sistema. Se encuentra asociada a la tabla `roles` y sus atributos son:

- `private Long id;`
- `private String nombreRol;`

Su función es almacenar los tipos de acceso que puede tener un usuario.

```
package cl.instrumentum.service_auth.model;

import jakarta.persistence.*;
import lombok.Data;

@Entity
@Table(name = "roles")
@Data
public class Rol {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombreRol;
}
```

DTO de autenticación (AuthRequest)

La clase `cl.instrumentum.service_auth.dto.AuthRequest` corresponde a un Data Transfer Object (DTO) utilizado para recibir las credenciales enviadas por el cliente durante el proceso de login. Esta clase no representa una tabla de base de datos, sino un objeto

utilizado exclusivamente para transportar información.

```
package cl.instrumentum.service_auth.dto;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class AuthRequest {

    private String nombreUsuario;
    private String contraseña;
}
```

Contiene:

- private String nombreUsuario;
- private String contraseña;

Su objetivo es:

- Separar los datos recibidos desde la API de las entidades del sistema.
- Evitar exponer directamente el modelo Usuario.
- Estructurar la información enviada al endpoint:

POST /auth/login

Ejemplo de solicitud:

```
{
  "nombreUsuario": "usuario",
  "contrasena": "password"
}
```

7. Implementación por Capas

El microservicio de autenticación del sistema Instrumentum sigue una arquitectura por capas, separando responsabilidades para mejorar la organización, escalabilidad y mantenibilidad del código.

Las capas implementadas son:

- Model
- Repository
- Service
- Controller
- Config

A continuación, se describe cada una aplicada al servicio de usuarios

Model

La capa **Model** se encarga de representar las entidades del sistema y su relación con la base de datos.

En el microservicio de autenticación se utilizan principalmente las entidades:

- Usuario
- Rol

```
15
16  @Entity
17  @Table(name = "banda")
18  @Data
19  @AllArgsConstructor
20  @NoArgsConstructor
21  public class Banda {
22
23      @Id
24      @GeneratedValue(strategy = GenerationType.IDENTITY)
25      private Long idBanda;
26
27      @NotBlank(message = "El nombre de la banda es obligatorio")
28      private String nombre;
29
30      @NotNull(message = "La fecha de registro es obligatoria")
31      private LocalDate fechaRegistro;
32  }
```

```

@Entity
@Table(name = "usuario")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idUser;

    @NotBlank(message = "El username es obligatorio")
    @Size(max = 30, message = "El username no puede superar los 30 caracteres")
    @Column(unique = true)
    private String username;

    @NotBlank(message = "El email es obligatorio")
    @Email(message = "El correo electrónico no tiene un formato válido")
    private String email;

    @NotBlank(message = "El rol es obligatorio")
    @Pattern(
        regexp = "Musico|Tech",
        message = "El rol debe ser Musico o Tech"
    )
    private String rol;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "id_banda")
    private Banda banda;
}

```

Estas clases permiten mapear la estructura de la base de datos utilizando JPA.

La entidad Usuario define la información principal del sistema, como nombre de usuario, contraseña, correo electrónico y roles asignados.

La entidad Rol permite definir los permisos o niveles de acceso dentro del sistema.

Repository

Responsabilidad

La capa **Repository** del microservicio de usuarios es la encargada de gestionar el acceso a la base de datos mediante Spring Data JPA, permitiendo realizar operaciones de consulta y persistencia sobre las entidades del sistema sin necesidad de implementar consultas SQL manuales. En este microservicio se utilizan dos repositorios principales:

UsuarioRepository

El repositorio UsuarioRepository gestiona la entidad Usuario, la cual representa a los usuarios registrados en el sistema.

Responsabilidades:

- Realizar operaciones CRUD sobre la entidad Usuario.
- Buscar usuarios por su nombre de usuario.
- Obtener usuarios asociados a una banda específica.
- Ejecutar consultas personalizadas mediante JPQL.

```
@Repository
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {

    Usuario findByUsername(String username);

    @Query("""
        SELECT u
        FROM Usuario u
        WHERE u.banda.idBanda = :idBanda
        """)
    List<Usuario> findAllByBandaId(@Param("idBanda") Long idBanda);
}
```

Métodos implementados:

- Usuario findByUsername(String username);
 - Permite obtener un usuario a partir de su nombre de usuario, utilizado para validaciones internas del sistema.
- List<Usuario> findAllByBandaId(@Param("idBanda") Long idBanda);
 - Permite obtener todos los usuarios asociados a una banda específica, utilizando una consulta JPQL personalizada.

BandaRepository

El repositorio BandaRepository gestiona la entidad Banda, la cual representa la agrupación musical a la que pertenecen los usuarios.

```
package cl.instrumentum.service_usuario.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import cl.instrumentum.service_usuario.model.Banda;

@Repository
public interface BandaRepository extends JpaRepository<Banda, Long> {
    Banda findByNombre(String nombre);
}
```

Responsabilidades:

- Realizar operaciones CRUD sobre la entidad Banda.
- Buscar bandas por su nombre.
- Gestionar la persistencia de información relacionada con agrupaciones musicales.

Método implementado:

- Banda findByNombre(String nombre);
 - Permite buscar una banda a partir de su nombre.

Métodos heredados de JpaRepository:

Al extender JpaRepository<(Usuario/Banda), Long> se habilitan automáticamente los siguientes métodos sin necesidad de implementación:

- save(Usuario entity) -> Guardar o actualizar un usuario.
- findById(Long id) -> Buscar un usuario por su ID.
- findAll() -> Obtener todos los usuarios.
- deleteById(Long id) -> Eliminar un usuario por su ID.
- delete(Usuario entity) -> Eliminar una entidad específica.
- existsById(Long id) -> Verificar si un usuario existe.
- count() -> Contar la cantidad de registros.

Service

La capa Service del microservicio de usuarios es la encargada de contener la lógica de negocio relacionada con la gestión de usuarios dentro del sistema. Esta capa actúa como intermediaria entre el controlador y el repositorio, procesando la información antes de ser almacenada o devuelta al cliente.

El servicio principal del módulo de usuarios se encarga de la administración de los datos de usuario, asegurando la correcta manipulación de la información y aplicando las reglas de negocio definidas para el sistema.

Responsabilidades principales

- Crear nuevos usuarios en el sistema.
- Obtener información de usuarios registrados.
- Actualizar datos de usuarios existentes.
- Eliminar usuarios del sistema.
- Coordinar la comunicación con la capa Repository.
- Aplicar reglas de negocio relacionadas con la gestión de usuarios.

Flujo general de operaciones

Registro de usuario

- El sistema recibe los datos del usuario/banda desde el controlador.
- Se validan los datos recibidos.
- Se envía la información al repositorio para su almacenamiento.
- Se retorna una confirmación de creación.

Consulta de usuarios

- Se solicita información desde el controlador.
- El servicio consulta el repositorio.
- Se retorna la lista de usuarios o bandas completa

Actualización de usuario

- Se recibe la solicitud con los nuevos datos.
- Se valida la existencia del usuario/banda.
- Se actualizan los campos correspondientes.
- Se guarda la información en la base de datos.

Eliminación de usuario

- Se verifica la existencia del usuario/banda.
- Se elimina el registro de la base de datos.
- Se confirma la operación al controlador.

```

@Service
public class UsuarioService {

    @Autowired
    private UsuarioRepository usuarioRepository;

    @Autowired
    private BandaRepository bandaRepository;

    @Autowired
    private WebClient.Builder webClientBuilder;

    public List<Usuario> listarUsuarios() {
        return usuarioRepository.findAll();
    }

    public Optional<Usuario> buscarUsuarioPorId(Long id) {
        return usuarioRepository.findById(id);
    }

    public Usuario guardarUsuario(Usuario usuario) {
        if (usuario.getBanda() != null && usuario.getBanda().getIdBanda() != null) {
            Banda banda = bandaRepository.findById(usuario.getBanda().getIdBanda())
                .orElseThrow(() -> new RuntimeException(message: "La banda no existe."));
            usuario.setBanda(banda);
        }
        return usuarioRepository.save(usuario);
    }

    // MODIFICADO: Retorna boolean verificando existencia
    public boolean eliminarUsuario(Long id) {
        if (usuarioRepository.existsById(id)) {
            usuarioRepository.deleteById(id);
            return true;
        }
        return false;
    }

```

```

    public List<Usuario> usuariosPorBanda(Long idBanda) {
        return usuarioRepository.findAllByBandaId(idBanda);
    }

    public List<Banda> listarBandas() {
        return bandaRepository.findAll();
    }

    public Optional<Banda> buscarBandaPorId(Long id) {
        return bandaRepository.findById(id);
    }

    public Banda guardarBanda(Banda banda) {
        return bandaRepository.save(banda);
    }

    // MODIFICACION 1: Retorna boolean verificando existencia
    // MODIFICACION 2: Verifica usuarios asociados antes de eliminar
    public boolean eliminarBanda(Long id) {
        if (bandaRepository.existsById(id)) {
            // 1. Regla de negocio: No borrar si hay usuarios adentro
            List<Usuario> usuariosAsociados = usuarioRepository.findAllByBandaId(id);
            if (!usuariosAsociados.isEmpty()) {
                throw new RuntimeException("No se puede eliminar la banda porque tiene " + usuariosAsociados.size() +
                    " usuarios asociados.");
            }

            // =====
            // LIMPIEZA MANUAL CON WEBCLIENT
            // =====

            // Paso 2: Limpiar Eventos
            try {
                webClientBuilder.build().delete()
                    .uri("http://localhost:8086/api/v2/eventos/banda/" + id)

```

```

        .uri("http://localhost:8086/api/v2/eventos/banda/" + id)
        .retrieve().toBodilessEntity().block();
    } catch (Exception e) {
        System.out.println("Ignorando error en Eventos: " + e.getMessage());
    }

    // Paso 3: Limpiar Finanzas
    try {
        webClientBuilder.build().delete()
            .uri("http://localhost:8087/api/v2/finanzas/banda/" + id)
            .retrieve().toBodilessEntity().block();
    } catch (Exception e) {
        System.out.println("Ignorando error en Finanzas: " + e.getMessage());
    }

    // Paso 4: Limpiar Giras
    try {
        webClientBuilder.build().delete()
            .uri("http://localhost:8088/api/v2/giras/banda/" + id)
            .retrieve().toBodilessEntity().block();
    } catch (Exception e) {
        System.out.println("Ignorando error en Giras: " + e.getMessage());
    }

    // Paso 5: Limpiar Logística
    try {
        webClientBuilder.build().delete()
            .uri("http://localhost:8090/api/v2/logistica/banda/" + id)
            .retrieve().toBodilessEntity().block();
    } catch (Exception e) {
        System.out.println("Ignorando error en Logística: " + e.getMessage());
    }

```

```

    // Paso 6: Limpiar Merchandising
    try {
        webClientBuilder.build().delete()
            .uri("http://localhost:8091/api/v2/merchandising/banda/" + id)
            .retrieve().toBodilessEntity().block();
    } catch (Exception e) {
        System.out.println("Ignorando error en Merchandising: " + e.getMessage());
    }

    // Paso 7: Limpiar Canciones (Rig Builder)
    try {
        webClientBuilder.build().delete()
            .uri("http://localhost:8085/api/v2/canciones/banda/" + id)
            .retrieve().toBodilessEntity().block();
    } catch (Exception e) {
        System.out.println("Ignorando error en Rig: " + e.getMessage());
    }

    // =====

    // Paso Final: Borrar la banda localmente
    bandaRepository.deleteById(id);
    return true;
}
return false;

```

Controller

La capa Controller del microservicio de usuarios es la encargada de exponer los endpoints REST que permiten la comunicación entre el cliente y el sistema.

Su función principal es recibir las solicitudes HTTP, delegarlas a la capa Service y retornar las respuestas correspondientes, manejando tanto operaciones exitosas como errores del sistema.

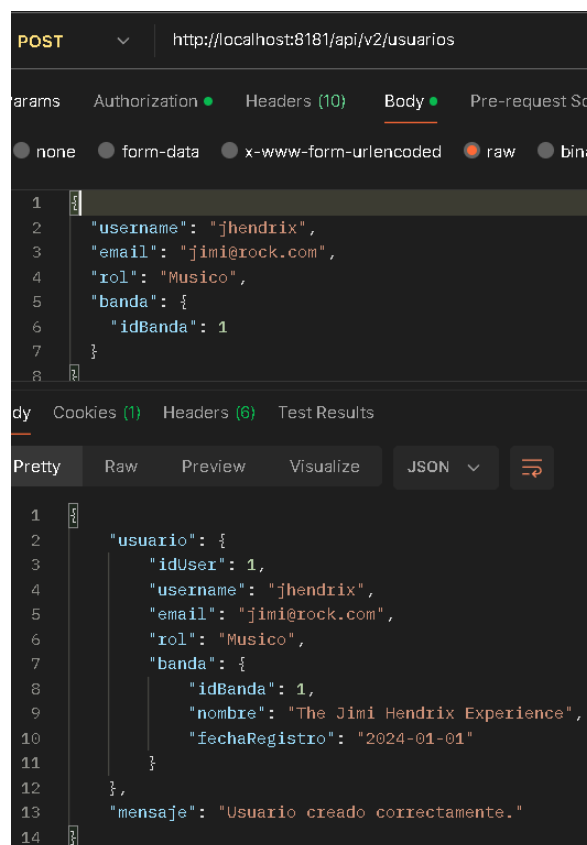
En este microservicio existen dos controladores principales

UsuarioController

Este controlador gestiona todas las operaciones relacionadas con la entidad Usuario.

Endpoints principales

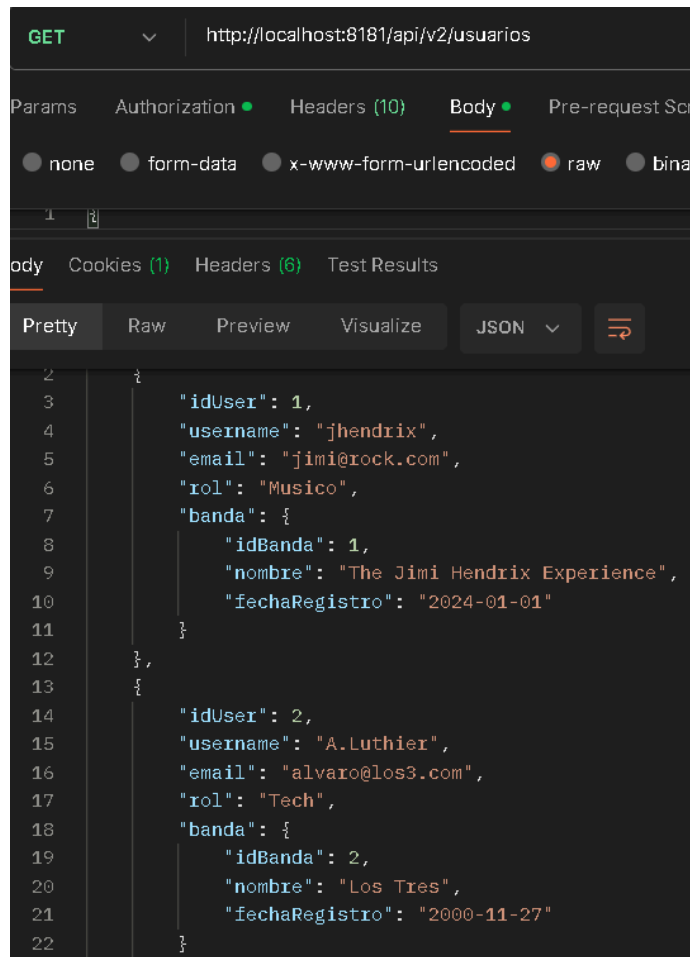
Crear usuario



```
@PostMapping
public ResponseEntity<Map<String, Object>> crear(@Valid @RequestBody Usuario usuario) {
    Usuario nuevo = usuarioService.guardarUsuario(usuario);
    return ResponseEntity.status(HttpStatus.CREATED)
        .body(Map.of(k1: "mensaje", v1: "Usuario creado correctamente.", k2: "usuario", nuevo));
}
```

Permite registrar un nuevo usuario en el sistema, incluye validación de datos mediante @Valid en el model

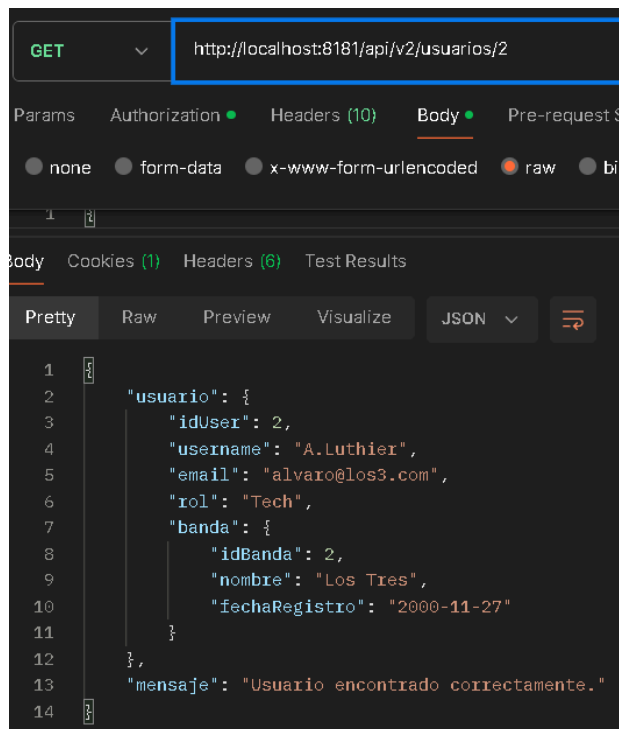
Listar usuarios



```
@GetMapping
public List<Usuario> listar() {
    return usuarioService.listarUsuarios();
}
```

Retorna la lista completa de usuarios registrados en el sistema.

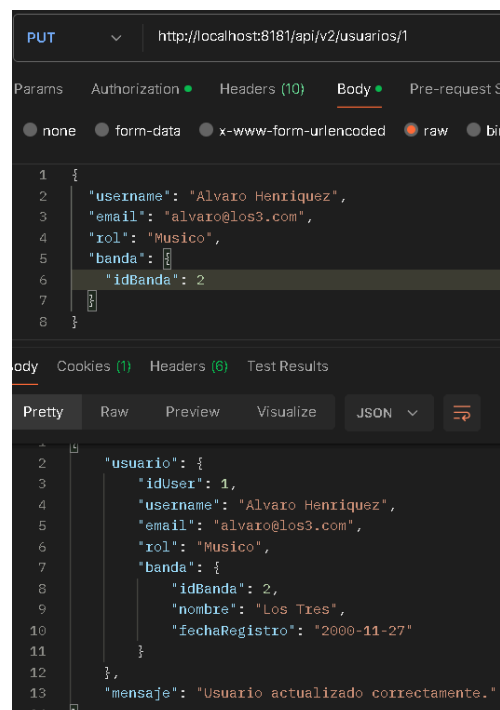
Obtener usuario por ID



```
@GetMapping("/{id}")
public ResponseEntity<Map<String, Object>> obtener(@PathVariable Long id) {
    return usuarioService.buscarUsuarioPorId(id)
        .map(u -> ResponseEntity.ok(
            Map.<String, Object>of(k1: "mensaje", v1: "Usuario encontrado correctamente.", k2: "usuario", u)))
        .orElse(ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.<String, Object>of(k1: "mensaje", "No existe un usuario con ID " + id + ".")));
}
```

Permite obtener la información de un usuario específico según su identificador.

Actualizar usuario




```

@PutMapping("/{id}")
public ResponseEntity<Map<String, Object>> actualizar(
    @PathVariable Long id,
    @Valid @RequestBody Usuario usuario) {

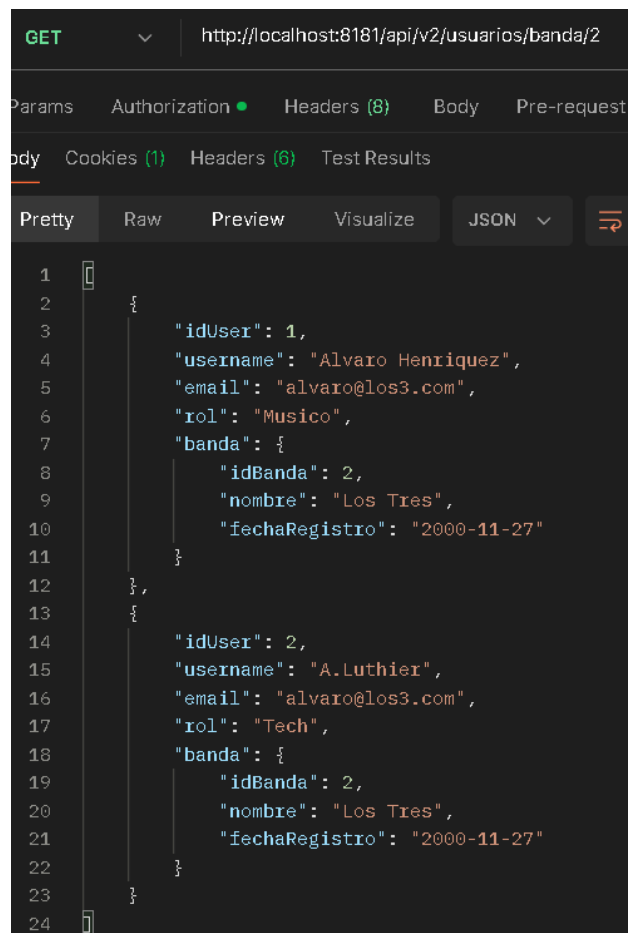
    return usuarioService.buscarUsuarioPorId(id)
        .map(existente -> {
            usuario.setIdUser(id);
            Usuario actualizado = usuarioService.guardarUsuario(usuario);
            return ResponseEntity.ok(
                Map.<String, Object>of(k1: "mensaje", v1: "Usuario actualizado correctamente.", k2: "usuario", actualizado));
        })
        .orElse(ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.<String, Object>of(k1: "mensaje", v1: "No existe un usuario con ID " + id + ".")));
}

```

Permite actualizar la información de un usuario existente.

El sistema verifica la existencia del usuario antes de realizar la actualización.

Usuarios por banda



```

GET http://localhost:8181/api/v2/usuarios/banda/2

Params Authorization Headers (8) Body Pre-request
body Cookies (1) Headers (6) Test Results

Pretty Raw Preview Visualize JSON

1
2 {
3   "idUser": 1,
4   "username": "Alvaro Henriquez",
5   "email": "alvaro@los3.com",
6   "rol": "Musico",
7   "banda": {
8     "idBanda": 2,
9     "nombre": "Los Tres",
10    "fechaRegistro": "2000-11-27"
11  },
12 },
13 {
14   "idUser": 2,
15   "username": "A.Luthier",
16   "email": "alvaro@los3.com",
17   "rol": "Tech",
18   "banda": {
19     "idBanda": 2,
20     "nombre": "Los Tres",
21     "fechaRegistro": "2000-11-27"
22   }
23 }
24

```

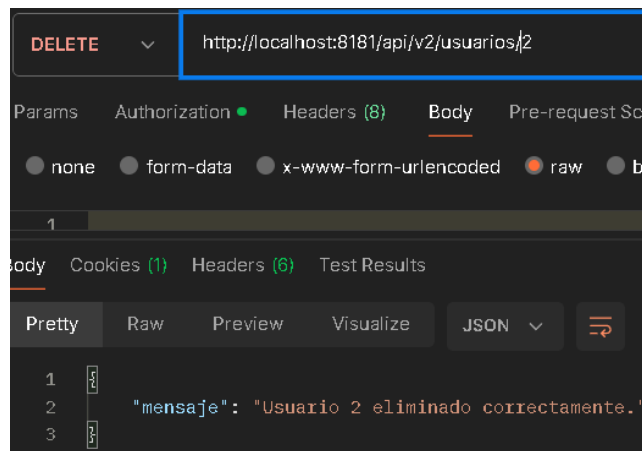
```

@GetMapping("/banda/{idBanda}")
public List<Usuario> porBanda(@PathVariable Long idBanda) {
    return usuarioService.usuariosPorBanda(idBanda);
}

```

Permite obtener todos los usuarios asociados a una banda específica.

Eliminar usuario



```
@DeleteMapping("/{id}")
public ResponseEntity<Map<String, String>> eliminar(@PathVariable Long id) {
    boolean eliminado = usuarioService.eliminarUsuario(id);

    if (eliminado) {
        return ResponseEntity.ok(Map.of(k1: "mensaje", "Usuario " + id + " eliminado correctamente."));
    } else {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.of(k1: "mensaje", "No existe un usuario con ID " + id + "."));
    }
}
```

Permite eliminar un usuario del sistema.

La respuesta depende del resultado de la operación realizada en el servicio.

Manejo de excepciones

El controlador incluye manejo global de errores mediante:

- RuntimeException
- MethodArgumentNotValidException

Esto permite:

- Capturar errores internos del sistema.
- Manejar errores de validación de datos.
- Retornar mensajes claros al cliente.

```
@ExceptionHandler(RuntimeException.class)
public ResponseEntity<Map<String, String>> handleRuntimeException(RuntimeException e) {
    String msg = e.getMessage() != null ? e.getMessage() : "Ocurrió un error inesperado.";
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(Map.of(k1: "mensaje", "Error interno del servidor: " + msg));
}

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, String>> handleValidationException(MethodArgumentNotValidException e) {
    String errores = e.getBindingResult().getFieldErrors()
        .stream()
        .map(fe -> fe.getField() + ": " + fe.getDefaultMessage())
        .collect(Collectors.joining(delimiter: ", "));
    return ResponseEntity.status(HttpStatus.BAD_REQUEST)
        .body(Map.of(k1: "mensaje", "Error de validación: " + errores));
}
```

Base de datos

Con todos los cambios anteriores, la tabla Usuario queda así:

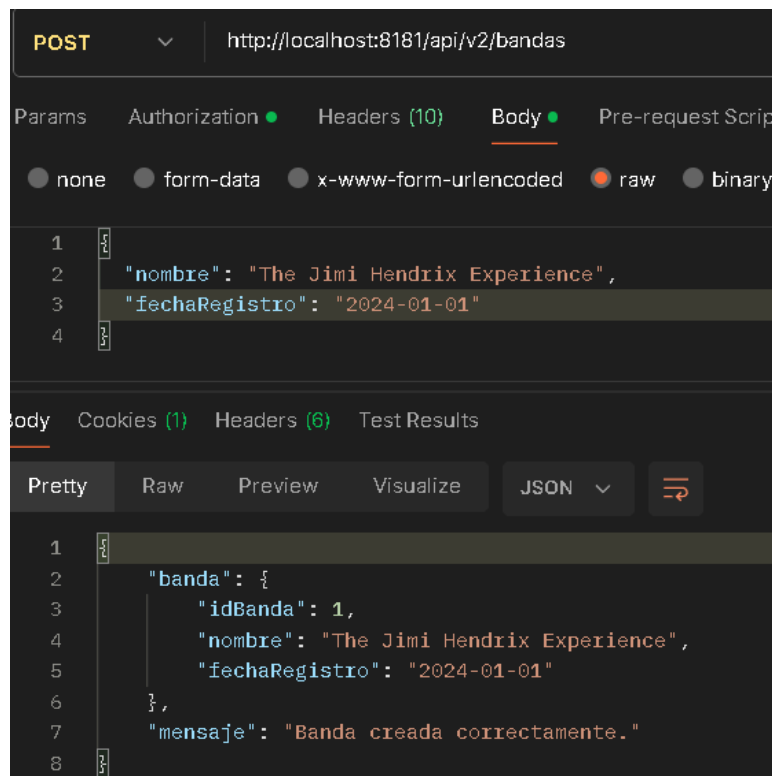
	id_banda	id_user	username	email	rol
☐ Editar ☐ Copiar ☐ Borrar	2	1	Alvaro Henriquez	alvaro@los3.com	Musico

BandaController

Este controlador gestiona las operaciones relacionadas con la entidad Banda.

Endpoints principales

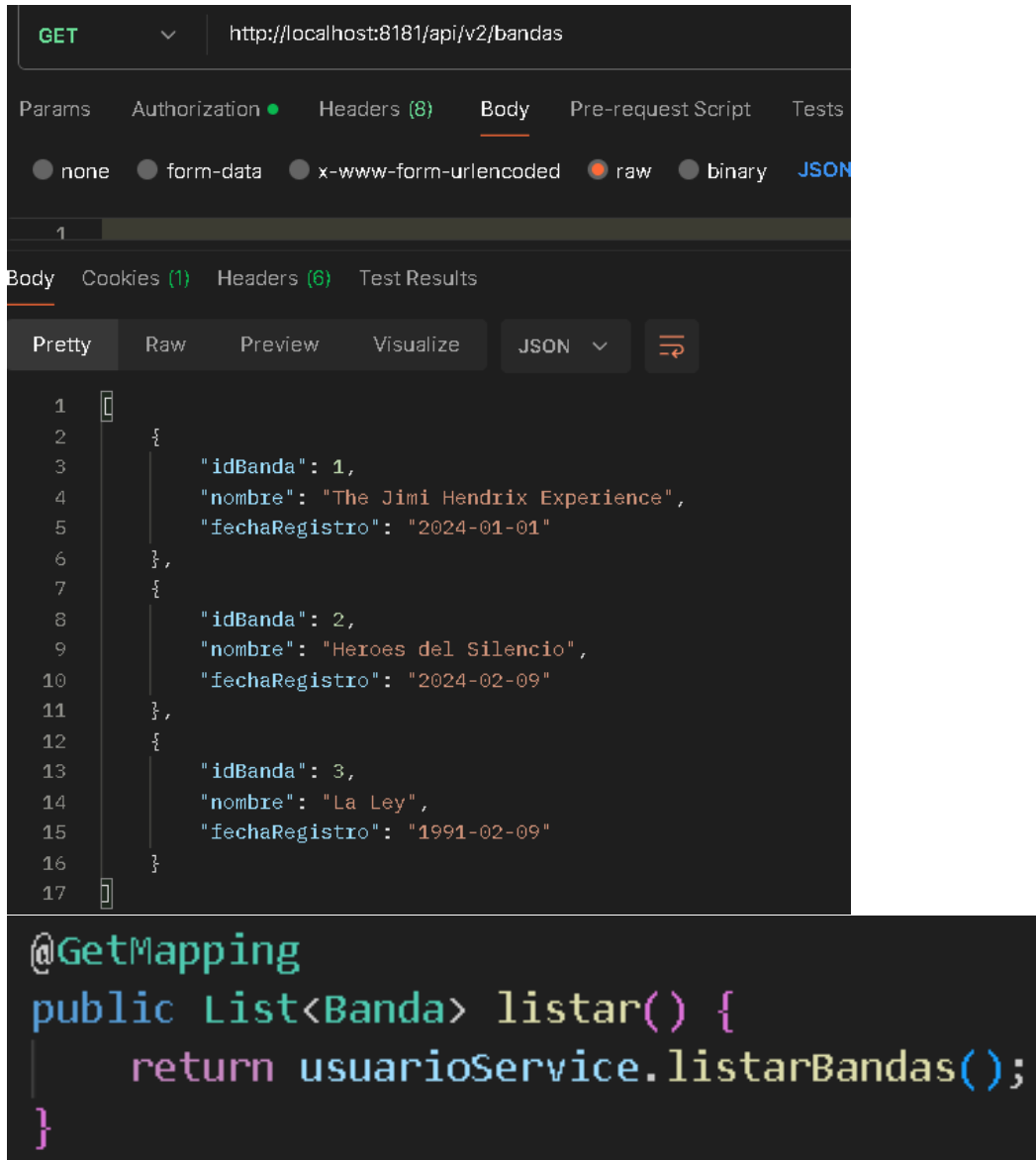
Crear banda



```
@PostMapping
public ResponseEntity<Map<String, Object>> crear(@Valid @RequestBody Banda banda) {
    Banda nueva = usuarioService.guardarBanda(banda);
    return ResponseEntity.status(HttpStatus.CREATED)
        .body(Map.<String, Object>of(k1: "mensaje", v1: "Banda creada correctamente.", k2: "banda", nueva));
}
```

Permite crear una banda con los atributos nombre y fecha de registro, puesto que el id es autogenerado y autoincremental

Listar bandas



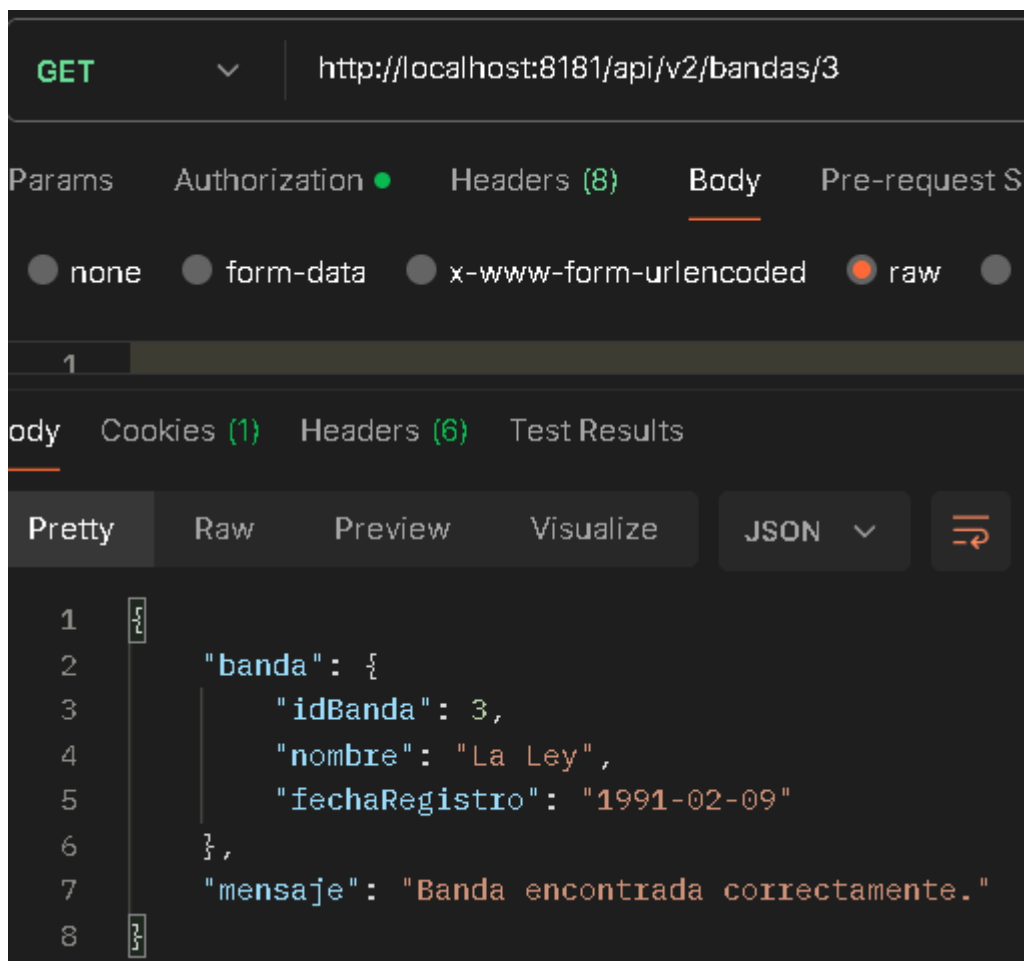
The screenshot shows a REST client interface with a GET request to `http://localhost:8181/api/v2/bandas`. The response is displayed in JSON format, showing a list of three bands. Below the screenshot, the corresponding Java code for the `listar` endpoint is shown.

```
1 {
2   {
3     "idBanda": 1,
4     "nombre": "The Jimi Hendrix Experience",
5     "fechaRegistro": "2024-01-01"
6   },
7   {
8     "idBanda": 2,
9     "nombre": "Heroes del Silencio",
10    "fechaRegistro": "2024-02-09"
11  },
12  {
13    "idBanda": 3,
14    "nombre": "La Ley",
15    "fechaRegistro": "1991-02-09"
16  }
17 }
```

```
@GetMapping
public List<Banda> listar() {
    return usuarioService.listarBandas();
}
```

Retorna todas las bandas registradas en el sistema.

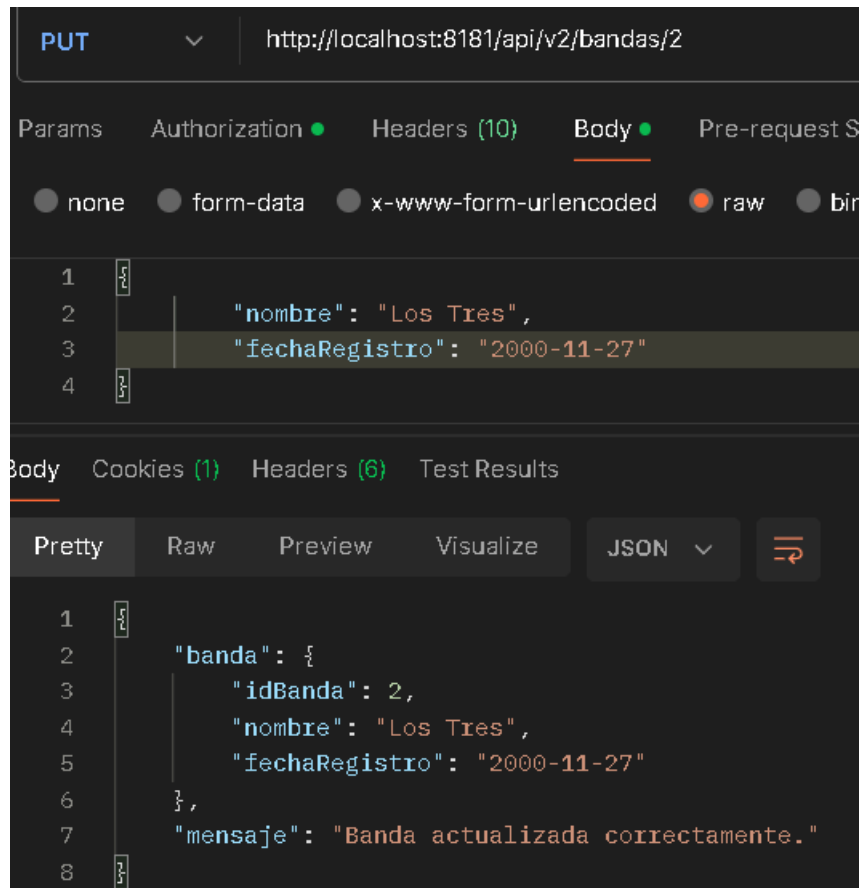
Obtener banda por ID



```
@GetMapping("/{id}")
public ResponseEntity<Map<String, Object>> obtener(@PathVariable Long id) {
    return usuarioService.buscarBandaPorId(id)
        .map(b -> ResponseEntity.ok(
            Map.<String, Object>of(k1: "mensaje", v1: "Banda encontrada correctamente.", k2: "banda", b)))
        .orElse(ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.<String, Object>of(k1: "mensaje", "No existe una banda con ID " + id + ".")));
}
```

Permite consultar una banda específica por su ID autoasignado.

Actualizar banda

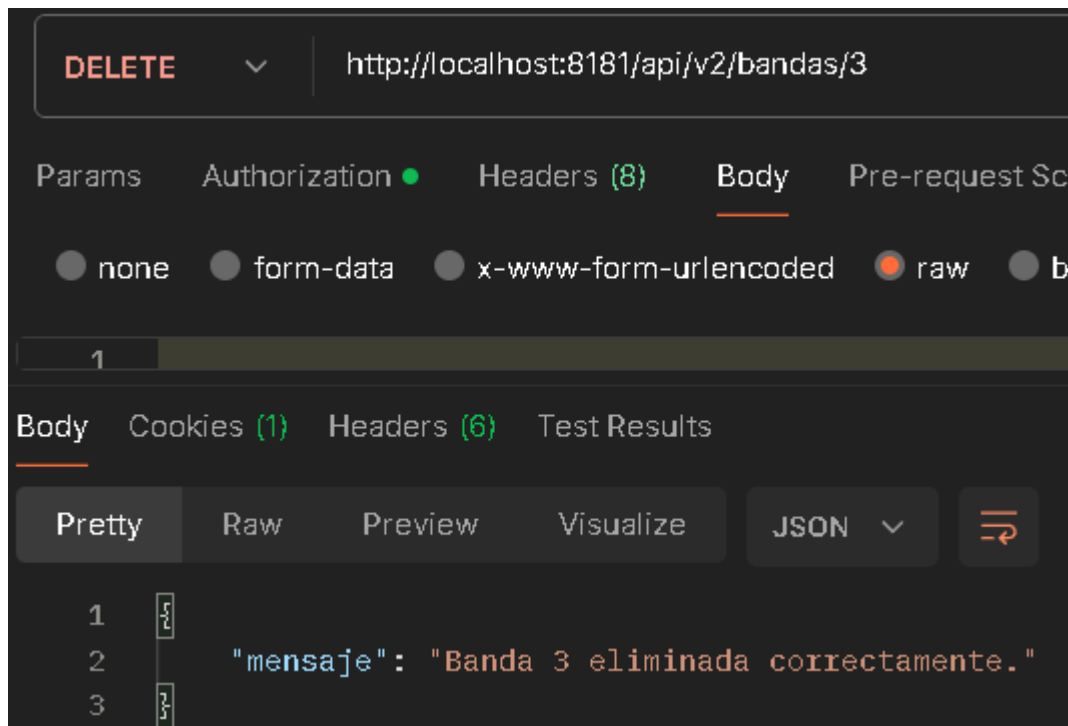


```
@PutMapping("/{id}")
public ResponseEntity<Map<String, Object>> actualizar(
    @PathVariable Long id,
    @Valid @RequestBody Banda banda) { // Escudo de validación activo

    return usuarioService.buscarBandaPorId(id)
        .map(existente -> {
            banda.setIdBanda(id); // Respetamos el ID de la URL
            Banda actualizada = usuarioService.guardarBanda(banda);
            return ResponseEntity.ok(
                Map.<String, Object>of(k1: "mensaje", v1: "Banda actualizada correctamente.", k2: "banda", actualizada));
        })
        .orElse(ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.<String, Object>of(k1: "mensaje", "No existe una banda con ID " + id + ".")));
}
```

Permite actualizar los datos de una banda existente.

Eliminar banda



```
@DeleteMapping("/{id}")
public ResponseEntity<Map<String, String>> eliminar(@PathVariable Long id) {
    boolean eliminado = usuarioService.eliminarBanda(id);

    if (eliminado) {
        return ResponseEntity.ok(Map.of(k1: "mensaje", "Banda " + id + " eliminada correctamente."));
    } else {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.of(k1: "mensaje", "No existe una banda con ID " + id + "."));
    }
}
```

Permite eliminar una banda del sistema.

Manejo de excepciones

Al igual que UsuarioController, incluye:

- Manejo de RuntimeException
- Manejo de validación con MethodArgumentNotValidException

```
@ExceptionHandler(RuntimeException.class)
public ResponseEntity<Map<String, String>> handleRuntimeException(RuntimeException e) {
    String msg = e.getMessage() != null ? e.getMessage() : "Ocurrió un error inesperado.";
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(Map.of(k1: "mensaje", "Error interno del servidor: " + msg));
}

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, String>> handleValidationException(MethodArgumentNotValidException e) {
    String errores = e.getBindingResult().getFieldErrors()
        .stream()
        .map(fe -> fe.getField() + ": " + fe.getDefaultMessage())
        .collect(Collectors.joining(delimiter: ", "));
    return ResponseEntity.status(HttpStatus.BAD_REQUEST)
        .body(Map.of(k1: "mensaje", "Error de validación: " + errores));
}
```

Base de datos

Con todos los cambios anteriores, la tabla Banda queda así:

				fecha_registro	id_banda	nombre
☐	 Editar	 Copiar	 Borrar	2024-01-01	1	The Jimi Hendrix Experience
☐	 Editar	 Copiar	 Borrar	2000-11-27	2	Los Tres

8. Comunicación entre Microservicios

Esta sección se redactará de acuerdo con la implementación real del proyecto.

Si la comunicación se realiza únicamente mediante el API Gateway, se explicará ese mecanismo.

Si existe comunicación directa entre servicios, se describirá la tecnología utilizada.

No se incluirán herramientas o librerías que no estén presentes en el código.

Figura 4. Comunicación entre microservicios.

(Adjuntar evidencia)

9. Swagger

¿Qué es?

Swagger es un conjunto de herramientas basadas en la especificación OpenAPI que permite documentar, visualizar y probar servicios REST de forma estandarizada. Su principal objetivo es facilitar la comprensión de los endpoints disponibles dentro de una aplicación, describiendo las rutas, métodos HTTP, parámetros, respuestas y modelos de datos utilizados por cada servicio.

Dentro de proyectos desarrollados con Spring Boot, Swagger puede integrarse mediante SpringDoc OpenAPI, permitiendo generar automáticamente la documentación a partir de las clases, controladores y anotaciones existentes en el código fuente.

¿En qué consiste?

La documentación generada mediante OpenAPI se construye automáticamente analizando los controladores REST presentes en cada microservicio.

Cuando la aplicación se encuentra en ejecución, SpringDoc genera un documento JSON que describe toda la API y posteriormente lo representa mediante Swagger UI, proporcionando una interfaz web interactiva. Esta interfaz permite:

- Visualizar los endpoints disponibles.
- Conocer los métodos HTTP implementados.
- Revisar parámetros de entrada.
- Consultar respuestas esperadas.
- Probar operaciones directamente desde el navegador.

Al tratarse de una generación automática, la documentación permanece sincronizada con el código implementado, reduciendo inconsistencias y mejorando el mantenimiento del proyecto.

Implementación

En Instrumentum se utilizó SpringDoc OpenAPI para generar la documentación de los microservicios que exponen APIs REST. La configuración se realizó incorporando las dependencias correspondientes dentro de cada servicio y utilizando anotaciones clave para enriquecer la interfaz:

- **@Tag:** Se aplica a nivel de Controlador para agrupar y categorizar lógicamente un conjunto de endpoints bajo un mismo título y descripción (por ejemplo, agrupar todas las rutas relacionadas con los shows bajo la etiqueta "Eventos").
- **@Operation:** Se utiliza sobre cada método (endpoint) específico para proporcionar un resumen claro (summary) y una descripción detallada (description) sobre la acción exacta que realiza dicha ruta.
- **@Schema:** Se aplica en los modelos de datos (Entidades) y en sus atributos para explicar qué representa cada campo, si es obligatorio, y proveer valores de ejemplo (example) que facilitan la comprensión de la estructura JSON esperada.

Se utilizarán tres microservicios como ejemplo para evidenciar esta implementación:

1. Microservicio de Eventos:

Este módulo expone los endpoints necesarios para gestionar la agenda de la banda, permitiendo operaciones CRUD completas y búsquedas específicas por agrupación.

Eventos Endpoints para la gestión integral de eventos musicales		
GET	/api/v2/eventos/{id}	Buscar evento por ID
PUT	/api/v2/eventos/{id}	Actualizar un evento existente
DELETE	/api/v2/eventos/{id}	Eliminar un evento
GET	/api/v2/eventos	Listar todos los eventos
POST	/api/v2/eventos	Crear un nuevo evento
GET	/api/v2/eventos/banda/{idBanda}	Obtener eventos por ID de Banda

```

@GetMapping("/{id}")
@Operation(summary = "Buscar evento por ID", description = "Recupera los detalles de un evento específico utilizando su identificador único.")
public ResponseEntity<Map<String, Object>> obtenerPorId(@PathVariable Long id) {
    return eventoService.buscarPorId(id)
        .map(e -> ResponseEntity.ok(
            Map.<String, Object>of(k1: "mensaje", v1: "Evento encontrado correctamente.", k2: "evento", e)))
        .orElse(ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.<String, Object>of(k1: "mensaje", "No existe un evento con ID " + id + ".")));
}

```

GET
/api/v2/eventos/{id}
Buscar evento por ID

Recupera los detalles de un evento específico utilizando su identificador único.

Parameters

Name	Description
id * required integer(\$int64) (path)	1

Execute

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8086/api/v2/eventos/1' \
-H 'accept: */*'

```

Request URL

```
http://localhost:8086/api/v2/eventos/1

```

Server response

Code	Details
200	Response body <pre>{ "mensaje": "Evento encontrado correctamente.", "evento": { "idEvento": 1, "idBanda": 1, "nombre": "Noche Acústica - Bar O'Higgins", "fecha": "2026-03-15", "canciones": "1,2" } }</pre> Response headers <pre>content-type: application/json</pre>

Responses

Code	Description
200	OK

Al interactuar con la documentación, Swagger permite ejecutar peticiones reales. En el siguiente ejemplo se realiza una consulta para obtener un evento por su identificador único.

2. Microservicio de Logística

En este módulo, la documentación agrupa de manera clara las operaciones relacionadas con la gestión de contenedores (baúles o *flightcases*) y el empaque del equipamiento musical dentro de ellos.

Logística Gestión de contenedores y empaque de equipos para shows		
GET	/api/v2/logistica/{id}	Obtener un contenedor por ID
PUT	/api/v2/logistica/{id}	Actualizar un contenedor
DELETE	/api/v2/logistica/{id}	Eliminar un contenedor
POST	/api/v2/logistica	Crear un nuevo contenedor
GET	/api/v2/logistica/{id}/equipos	Listar equipos de un contenedor
POST	/api/v2/logistica/{id}/equipos	Agregar equipo a un contenedor
GET	/api/v2/logistica/todos	Lista todos los contenedores
GET	/api/v2/logistica/todosEquipos	Listar todos los equipos con sus contenedores
GET	/api/v2/logistica/banda/{idBanda}	Listar contenedores de una banda
DELETE	/api/v2/logistica/{id}/equipos/{idEquipo}	Remover equipo de un contenedor
DELETE	/api/v2/logistica/equipos/{idEquipo}	Remover equipo globalmente

```
@Operation(summary = "Obtener un contenedor por ID", description = "Busca un contenedor específico en el sistema por su ID")
@GetMapping("/{id}")
public ResponseEntity<?> obtener(@PathVariable Long id) {
    return logisticaService.buscarPorId(id)
        .<ResponseEntity<?>>map(ResponseEntity::ok)
        .orElseGet(() -> ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body("Error: Contenedor no encontrado"));
}
```

GET

/api/v2/logistica/{id}

Obtener un contenedor por ID

Busca un contenedor específico en el sistema por su ID

Parameters

Name	Description
id * required	
integer(\$int64)	1
(path)	

Execute

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8090/api/v2/logistica/1' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8090/api/v2/logistica/1
```

Server response

Code	Details
200	Response body <pre>{ "idContenedor": 1, "idBanda": 1, "nombreCaja": "Flightcase Principal", "peso": 12.5, "equipos": [{ "id": 1, "idEquipo": 1 }, { "id": 2, "idEquipo": 4 }] }</pre>

A través de Swagger, es posible comprobar cómo el sistema devuelve estructuras de datos complejas de forma legible.

3. Microservicio de Especificaciones (Specs)

Este servicio demuestra cómo la API documenta y separa rutas especializadas para un mismo módulo, dividiendo los endpoints según se trate de especificaciones para instrumentos de madera o equipos electrónicos.

Specs Endpoints para la gestión de especificación de equipos		
GET	/api/v2/especs/instrumento/{equipoId}	Obtener especificación de instrumento por ID
PUT	/api/v2/especs/instrumento/{equipoId}	Actualizar especificación de instrumento
POST	/api/v2/especs/instrumento/{equipoId}	Crear especificación de instrumento
GET	/api/v2/especs/electronica/{equipoId}	Obtener especificación electrónica
PUT	/api/v2/especs/electronica/{equipoId}	Actualizar especificación electrónica
POST	/api/v2/especs/electronica/{equipoId}	Crear especificación electrónica
DELETE	/api/v2/especs/equipo/{equipoId}	Eliminar especificaciones por equipo

```
@GetMapping("/{instrumento/{equipoId}}")
@Operation(summary = "Obtener especificación de instrumento por ID", description = "Recupera las especificaciones técnicas de un instrumento mediante el ID del equipo.")
public ResponseEntity<Map<String, Object>> obtenerInstrumento(@PathVariable Long equipoId) {
    return specsService.obtenerInstrumentoPorId(equipoId)
        .map(i -> {
            Map<String, Object> response = new HashMap<>();
            response.put(key: "mensaje", value: "Especificación de instrumento encontrada correctamente.");
            response.put(key: "especificacion", i);
            return ResponseEntity.ok(response);
        })
        .orElse(ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.of(k1: "mensaje", "No existen especificaciones de instrumento para el equipo con ID " + equipoId + ".")));
}
```

GET

/api/v2/especs/instrumento/{equipoId}

Obtener especificación de instrumento por ID

Recupera las especificaciones técnicas de un instrumento mediante el ID del equipo.

Parameters

Name	Description
equipoId * required	
integer(\$int64)	1
(path)	

Execute

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8083/api/v2/especs/instrumento/1' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8083/api/v2/especs/instrumento/1
```

Server response

Code	Details
200	Response body <pre>{ "mensaje": "Especificación de instrumento encontrada correctamente.", "especificacion": {</pre>

Al consultar un equipo específico, la interfaz interactiva refleja de inmediato los datos técnicos definidos en el sistema.

Conclusión de la Implementación

La integración de SpringDoc OpenAPI y Swagger UI en el ecosistema de microservicios de Instrumentum representa un avance fundamental para la mantenibilidad y escalabilidad del proyecto. Al automatizar la generación de la documentación directamente desde el código fuente mediante anotaciones, se elimina la brecha histórica entre la implementación real y los manuales técnicos, garantizando que los contratos de la API nunca queden obsoletos.

Además, la provisión de una interfaz visual e interactiva agiliza drásticamente el flujo de trabajo. Transforma la documentación estática en una herramienta de pruebas en tiempo real, creando un entorno centralizado y estandarizado que facilita la comunicación técnica y permite a los desarrolladores probar, comprender y consumir los distintos endpoints de manera autónoma y eficiente.

10. Testing

10.1 ¿Qué es el testing?

El testing (o pruebas de software) es un proceso fundamental en el ciclo de vida del desarrollo que consiste en evaluar y verificar que una aplicación o sistema informático funciona exactamente como se espera. Se encarga de identificar errores, brechas de seguridad o comportamientos anómalos antes de que el software sea desplegado a un entorno de producción.

La importancia del testing radica en la mitigación de riesgos. En una arquitectura de microservicios como *Instrumentum*, donde múltiples componentes interactúan entre sí de forma constante, un fallo en un servicio no detectado puede propagarse y comprometer todo el sistema. El testing actúa como una red de seguridad que asegura la estabilidad estructural del código frente a futuros cambios o actualizaciones.

Beneficios del testing:

- **Detección temprana de errores:** Permite corregir *bugs* en fases de desarrollo, donde el costo de reparación es significativamente menor.
- **Refactorización segura:** Los desarrolladores pueden mejorar o limpiar el código con la tranquilidad de que, si rompen alguna funcionalidad existente, las pruebas fallarán y avisarán de inmediato.

- **Documentación viva:** Las pruebas bien escritas sirven como manual de instrucciones, mostrando a otros desarrolladores cómo se espera que interactúen las clases y métodos.
- **Calidad garantizada:** Mejora la experiencia del usuario final al ofrecer un producto estable y predecible.

10.2 Pruebas unitarias

Las pruebas unitarias son un nivel de testing que aísla y evalúa la porción más pequeña de código ejecutable (generalmente un método o una clase) para comprobar su correcto funcionamiento de manera independiente del resto del sistema. Para lograr este aislamiento en *Instrumentum*, se utilizó la librería **Mockito**, la cual permite simular (mockear) el comportamiento de las bases de datos (Repositorios) y las llamadas externas (WebClient).

Explicación del patrón AAA: Para mantener el código de las pruebas limpio y estandarizado, se utilizó el patrón AAA, que divide cada prueba en tres fases lógicas:

- **Arrange (Preparar):** Se configuran las condiciones iniciales de la prueba. Se instancian los objetos necesarios (mocks), se definen los parámetros de entrada y se instruye a las dependencias simuladas sobre qué deben responder cuando sean llamadas (uso de `when(...).thenReturn(...)`).
- **Act (Actuar):** Se ejecuta la acción central que se desea evaluar. Consiste en una única llamada al método del servicio correspondiente, guardando su retorno en una variable para su posterior análisis.
- **Assert (Afirmar):** Se comprueba que el resultado obtenido en la fase de "Actuar" coincida con el resultado esperado. Se utilizan aserciones (como `assertEquals`, `assertNotNull`, `assertTrue`) y verificaciones (`verify`) para asegurar que el sistema interactuó correctamente con sus dependencias.

10.3 Documentación de Pruebas Implementadas (Microservicios)

A continuación, se documentan las pruebas unitarias implementadas en 6 de los microservicios centrales de la plataforma:

1. Microservicio: Finanzas

Método probado: listarTransaccionesTest

- **Objetivo:** Verificar que el servicio es capaz de recuperar y devolver correctamente la lista completa de transacciones financieras almacenadas en el sistema.
- **Resultado esperado:** Se espera que el método no devuelva valores nulos, que el tamaño de la lista coincida con los datos simulados (tamaño 1) y que se llame exactamente una vez al método findAll() del repositorio.
- **Evidencia:**

```
@Test
public void listarTransaccionesTest() {
    // Arrange
    when(transaccionRepository.findAll()).thenReturn(List.of(transaccionBase));

    // Act
    List<Transaccion> resultado = transaccionService.listarTransacciones();

    // Assert
    assertNotNull(resultado);
    assertEquals(expected: 1, resultado.size());
    verify(transaccionRepository).findAll();
}
```

Método probado: guardarTransaccionTest

- **Objetivo:** Comprobar que una transacción financiera es procesada y guardada exitosamente, simulando previamente un flujo feliz de validación a través de WebClient.
- **Resultado esperado:** El método debe retornar la transacción registrada, validando que el tipo de movimiento ("ingreso") se mantenga intacto y que el repositorio haya ejecutado la acción save().

- **Evidencia:**

```
@rest
public void guardarTransaccionTest() {
    // Arrange
    mockWebClientFlujoFeliz();
    when(transaccionRepository.save(any(Transaccion.class))).thenReturn(transaccionBase);

    // Act
    Transaccion resultado = transaccionService.guardarTransaccion(transaccionBase);

    // Assert
    assertNotNull(resultado);
    assertEquals(expected: "ingreso", resultado.getTipoMovimiento());
    verify(transaccionRepository).save(transaccionBase);
}
```

- **Resultados:**

✓	🔍	TransaccionServiceTest	7.0s
✓	🔍	listarTransaccionesTest()	77ms
✓	🔍	buscarTransaccionPorIdTest()	44ms
✓	🔍	guardarTransaccionTest()	76ms
✓	🔍	actualizarTransaccionTest()	54ms
✓	🔍	eliminarTransaccionTest()	326ms
✓	🔍	obtenerTransaccionesPorBandaTest()	6.4s

2. Microservicio: Giras

Método probado: guardarParadaTest

- **Objetivo:** Evaluar la lógica de almacenamiento de una nueva escala logística (Parada) dentro de una Gira existente.
- **Resultado esperado:** El sistema debe encontrar la gira madre y persistir la parada. Se verifica que el objeto retornado contenga la ciudad correcta ("Santiago") y se ejecuten las validaciones en los repositorios asociados.
- **Evidencia:**

```
@Test
void guardarParadaTest() {
    when(giraRepository.findById(id: 1L)).thenReturn(Optional.of(giraMock));
    when(paradaRepository.save(any(ParadaGira.class))).thenReturn(paradaMock);

    ParadaGira resultado = giraService.guardarParada(paradaMock);

    assertNotNull(resultado);
    assertEquals(expected: "Santiago", resultado.getCiudad());
    verify(giraRepository, times(1)).findById(id: 1L);
    verify(paradaRepository, times(1)).save(paradaMock);
}
```


Método probado: listarGirasPorBandaTest

- **Objetivo:** Validar que el sistema pueda filtrar correctamente el historial de giras asociadas exclusivamente al identificador de una agrupación musical específica.
- **Resultado esperado:** Se retorna una lista de giras donde el atributo idBanda del primer elemento coincide exactamente con el ID solicitado (10L), confirmando la efectividad del filtro.
- **Evidencia:**

```
@Test
void listarGirasPorBandaTest() {
    List<Gira> lista = List.of(giraMock);
    when(giraRepository.findByIdBanda(idBanda: 10L)).thenReturn(lista);

    List<Gira> resultado = giraService.listarPorBanda(idBanda: 10L);

    assertNotNull(resultado);
    assertEquals(expected: 1, resultado.size());
    assertEquals(expected: 10L, resultado.get(index: 0).getIdBanda());
    verify(giraRepository, times(1)).findByIdBanda(idBanda: 10L);
}
```

- Resultados:

GiraServiceTest 392ms		ParadaGiraServiceTest 19.6s	
✓	listarGirasTest()	✓	listarParadasPorGiraTest()
✓	listarGirasPorBandaTest()	✓	buscarParadaPorIdTest()
✓	buscarGiraPorIdTest()	✓	guardarParadaTest()
✓	guardarGiraTest()	✓	actualizarParadaTest()
✓	actualizarGiraTest()	✓	eliminarParadaTest()
✓	eliminarGiraTest()		

3. Microservicio: Logística

Método probado: listarPorBandaTest

- **Objetivo:** Asegurar que el sistema logístico recupera correctamente todos los contenedores (flightcases) pertenecientes a una banda tras validar la existencia de dicha agrupación.
- **Resultado esperado:** La prueba debe simular exitosamente la respuesta del WebClient, devolver una lista no nula y asegurar que el contenedor retornado pertenece a la banda consultada.

- **Evidencia:**

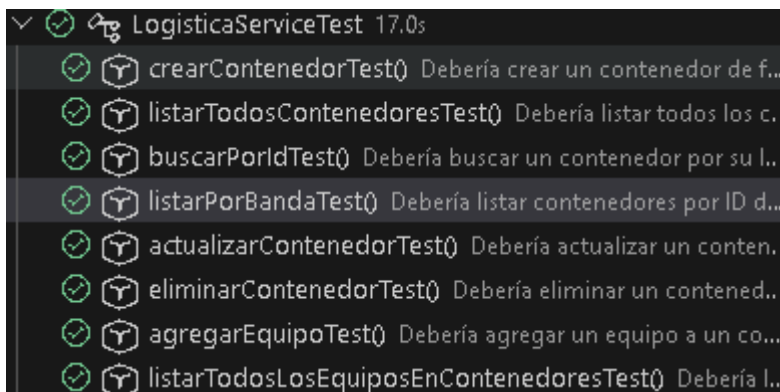
```
@Test
@DisplayName("Debería listar contenedores por ID de banda de forma exitosa")
void listarPorBandaTest() {
    mockWebClientExitoso();
    when(responseSpec.toBodilessEntity()).thenReturn(Mono.empty()); // Validación de banda OK

    List<Contenedor> listaMock = new ArrayList<>();
    listaMock.add(new Contenedor(idContenedor: 1L, idBanda: 10L, nombreCaja: "Flightcase Guitarras", peso: 25);
    when(contenedorRepository.findByIdBanda(idBanda: 10L)).thenReturn(listaMock);

    List<Contenedor> resultado = logisticaService.listarPorBanda(idBanda: 10L);

    assertNotNull(resultado);
    assertEquals(expected: 1, resultado.size());
    assertEquals(expected: 10L, resultado.get(index: 0).getIdBanda());
    verify(contenedorRepository, times(1)).findByIdBanda(idBanda: 10L);
}
```

- **Resultados:**



4. Microservicio: Merchandising

Método probado: listarVentasPorProductoTest

- **Objetivo:** Evaluar el método encargado de consultar el historial comercial y de transacciones asociadas a un producto de mercancía específico (ej. Polera Tour 2026).
- **Resultado esperado:** Se debe recuperar una lista no nula de transacciones, con un tamaño exacto igual a los elementos simulados en el *Arrange*, y verificar que el repositorio ejecutó la búsqueda usando el objeto producto correspondiente.

- **Evidencia:**

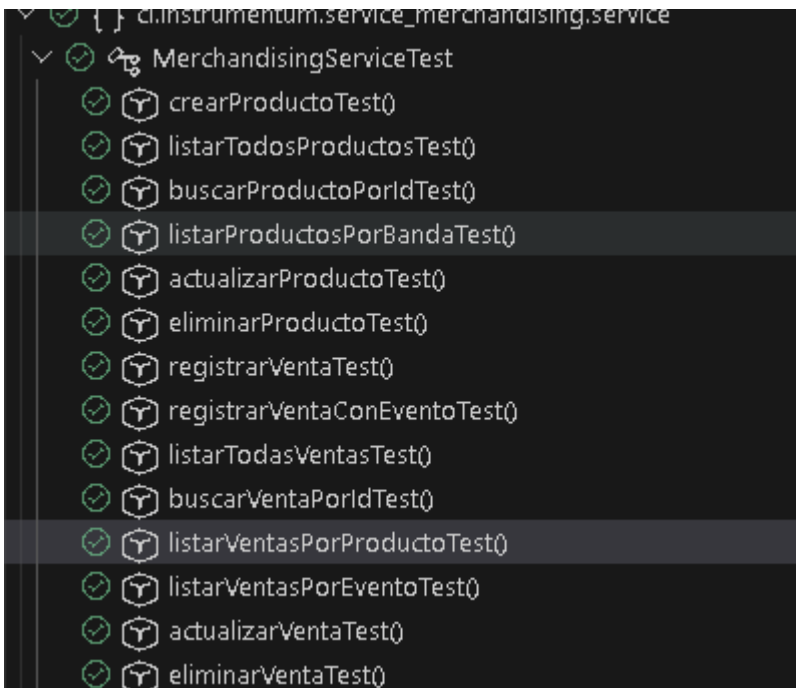
```
@Test
@DisplayName("Debería listar las ventas de un producto de forma exitosa")
void listarVentasPorProductoTest() {
    ProductoMerch p = new ProductoMerch(idProducto: 1L, idBanda: 1L, nombre: "Polera Tour 2026", tipo: "Polera");
    when(productoMerchRepository.findById(id: 1L)).thenReturn(Optional.of(p));

    List<VentaMerch> listaMock = new ArrayList<>();
    listaMock.add(new VentaMerch(idVenta: 1L, p, cantidad: 2, montoTotal: 30000.0, idEventoOrigen: null));
    when(ventaMerchRepository.findByProducto(p)).thenReturn(listaMock);

    List<VentaMerch> resultado = merchandisingService.listarVentasPorProducto(idProducto: 1L);

    assertNotNull(resultado);
    assertEquals(expected: 1, resultado.size());
    verify(ventaMerchRepository, times(1)).findByProducto(p);
}
```

- **Resultados:**



5. Microservicio: Mantenimiento

Método probado: registrarMantenimientoTest

- **Objetivo:** Probar el registro de una nueva bitácora de servicio técnico para un equipo, simulando de manera encadenada la respuesta síncrona (.block()) del WebClient que verifica la existencia del instrumento.
- **Resultado esperado:** El mantenimiento se guarda satisfactoriamente asociando el ID del equipo correcto (105L), garantizando que las integraciones reactivas no bloqueen el hilo de ejecución principal.

- **Evidencia:**

```
@SuppressWarnings("unchecked")
@Test
void registrarMantenimientoTest() {
    // Arrange & Mocking encadenado de WebClient para simular flujo feliz síncrono (.block())
    when(webClientBuilder.build()).thenReturn(webClient);
    when(webClient.get()).thenReturn(requestHeadersUriSpec);
    when(requestHeadersUriSpec.uri(anyString())).thenReturn(requestHeadersSpec);
    when(requestHeadersSpec.retrieve()).thenReturn(responseSpec);
    when(responseSpec.toBodilessEntity()).thenReturn(Mono.empty());

    when(mantenimientoRepository.save(any(Mantenimiento.class))).thenReturn(mantenimientoMock);

    // Act
    Mantenimiento resultado = mantenimientoService.registrarMantenimiento(mantenimientoMock);

    // Assert
    assertNotNull(resultado);
    assertEquals(expected: 105L, resultado.getEquipoId());
    verify(mantenimientoRepository, times(1)).save(mantenimientoMock);
}
```

- **Resultados:**

✓	🔗	MantenimientoServiceTest	2.2s
✓	🔗	buscarMantenimientoPorIdTest()	2.1s
✓	🔗	registrarMantenimientoTest()	83ms
✓	🔗	listarMantenimientoPorEquipoTest()	13ms
✓	🔗	requiereMantenimientoTest()	10ms
✓	🔗	eliminarMantenimientoTest()	7.0ms
✓	🔗	eliminarMantenimientoPorIdTest()	12ms

6. Microservicio: Inventario

Método probado: crearMarcaTest

- **Objetivo:** Verificar la correcta creación y persistencia de un registro de marca (fabricante de equipamiento) en la base de datos de inventario.
- **Resultado esperado:** El objeto guardado no debe ser nulo y el nombre devuelto debe coincidir con los datos inyectados de prueba ("Gibson").

- **Evidencia:**

```
@Test
void crearEquipoTest() {
    mockWebClientForValidation();
    // SOLUCIÓN: Mockear el findById para que el servicio crea que el equipo ya existe o pasa la validación
    when(equipoRepository.findById(equipoSample.getId())).thenReturn(Optional.of(equipoSample));
    when(equipoRepository.findByNombre(equipoSample.getNombre())).thenReturn(Optional.of(equipoSample));
    when(equipoRepository.save(any(Equipo.class))).thenReturn(equipoSample);

    Equipo resultado = inventarioService.guardarEquipo(equipoSample);

    assertNotNull(resultado);
    assertEquals(expected: "Stratocaster", resultado.getNombre());
    verify(equipoRepository, times(1)).save(equipoSample);
}
```

Método probado: listarTodosEquiposTest & listarEquiposPorPropietarioTest

- **Objetivo:** Probar los endpoints de lectura del inventario general y los filtros específicos de pertenencia (equipos asignados a un propietarioId).
- **Resultado esperado:** Las aserciones (assertFalse(resultado.isEmpty()) y assertEquals(1, resultado.size())) deben pasar con éxito, demostrando que la consulta SQL en memoria simulada retorna los datos íntegros.
- **Evidencia:**

```
@Test
void listarTodosEquiposTest() {
    when(equipoRepository.findAll()).thenReturn(List.of(equipoSample));

    List<Equipo> resultado = inventarioService.listarTodos();

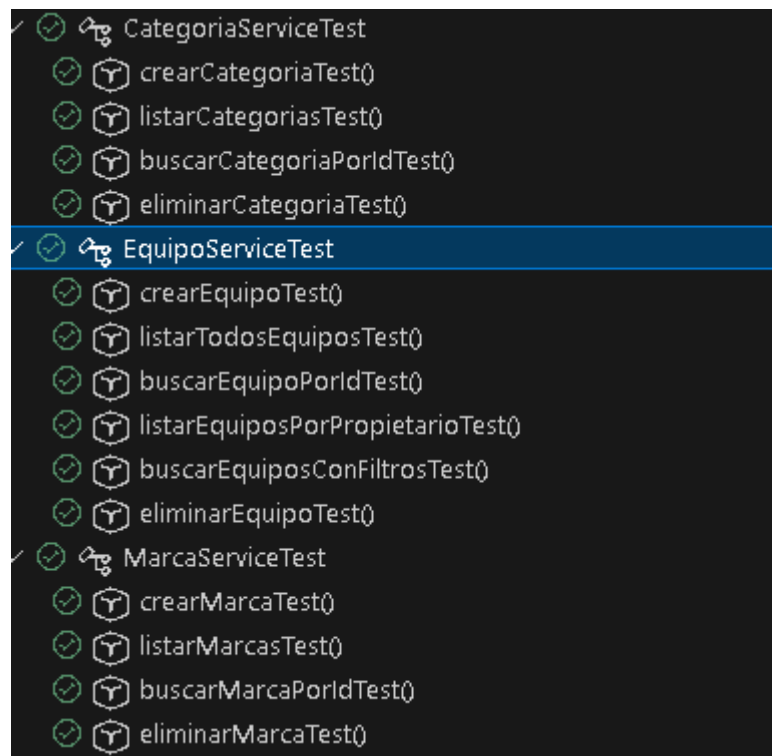
    assertFalse(resultado.isEmpty());
    assertEquals(expected: 1, resultado.size());
    verify(equipoRepository, times(1)).findAll();
}
```

```
@Test
void listarEquiposPorPropietarioTest() {
    when(equipoRepository.findByPropietarioId(propietarioId: 10L)).thenReturn(List.of(equipoSample));

    List<Equipo> resultado = inventarioService.listarEquiposPorPropietario(propietarioId: 10L);

    assertEquals(expected: 1, resultado.size());
    verify(equipoRepository, times(1)).findByPropietarioId(propietarioId: 10L);
}
```

- **Resultados:**



Resultados de Ejecución de los Tests Global

Una vez documentada la lógica y la estructura del patrón AAA en los microservicios, se procedió a la ejecución global de los *Test Suites*. Los entornos de pruebas de Spring Boot levantaron los contextos correspondientes y ejecutaron las validaciones.

11. Conclusiones

El desarrollo del sistema Instrumentum permitió implementar una plataforma orientada a la administración de equipamiento musical utilizando una arquitectura basada en microservicios con Spring Boot y MySQL. La separación del sistema en diferentes servicios permitió dividir las responsabilidades del proyecto en módulos independientes, facilitando su desarrollo, prueba y mantenimiento.

11.1 Arquitectura

La arquitectura basada en microservicios permitió separar los distintos dominios del sistema, como usuarios, inventario, eventos, giras, mantenimiento, logística, merchandising, finanzas y autenticación. Cada microservicio cuenta con sus propias capas de desarrollo (Model, Repository, Service y Controller), permitiendo una organización clara del código y evitando la dependencia directa entre módulos. Además, el uso de un API Gateway permite centralizar el acceso a los servicios, facilitando la comunicación entre clientes y microservicios.

11.2 Modularidad

La división del sistema en módulos independientes permitió que cada funcionalidad pueda ser desarrollada y modificada sin afectar directamente al resto de la aplicación. Cada microservicio representa una parte específica del negocio, permitiendo que nuevas funcionalidades puedan incorporarse de manera más ordenada. La modularidad también facilita las pruebas, ya que cada servicio puede ser evaluado de forma independiente.

11.3 Seguridad

La seguridad del sistema fue implementada mediante un servicio de autenticación basado en JWT. Este mecanismo permite validar la identidad de los usuarios mediante tokens generados después de un inicio de sesión exitoso, evitando mantener sesiones tradicionales en el servidor. Además, se aplicó encriptación de contraseñas utilizando BCrypt, protegiendo la información sensible almacenada en la base de datos.

11.4 Mantenibilidad

La aplicación de una arquitectura por capas permitió mejorar la mantenibilidad del código, separando responsabilidades entre controladores, servicios, repositorios y modelos. Esto facilita la detección de errores, modificación de funcionalidades y reutilización de componentes. La documentación mediante Swagger/OpenAPI también contribuye a la mantenibilidad, ya que permite conocer los endpoints disponibles y probar los servicios de manera visual. El uso de Git y GitHub como herramientas de control de versiones permitió

organizar el desarrollo mediante ramas independientes entre los desarrolladores, facilitando la colaboración y el seguimiento de cambios realizados en el proyecto.

11.5 Escalabilidad

El diseño basado en microservicios permite que el sistema pueda crecer de forma progresiva, ya que cada servicio puede evolucionar de manera independiente según las necesidades del negocio. Por ejemplo, un módulo con mayor carga de uso puede ser escalado sin necesidad de modificar o aumentar los recursos del resto de los servicios. Esta arquitectura permite que Instrumentum pueda incorporar nuevas funcionalidades futuras, nuevos módulos o mejoras tecnológicas manteniendo una estructura organizada y flexible.

12. Bibliografía

- Spring. (2026). *Spring Boot Reference Documentation*.
Disponible en:
<https://docs.spring.io/spring-boot/docs/current/reference/html/>
- Spring. (2026). *Spring Security Reference Documentation*.
Disponible en:
<https://docs.spring.io/spring-security/reference/>
- Spring. (2026). *Spring Data JPA Reference Documentation*.
Disponible en:
<https://docs.spring.io/spring-data/jpa/reference/>
- Oracle. (2026). *Java Documentation*.
Documentación oficial del lenguaje Java y plataforma JDK.
Disponible en:
<https://docs.oracle.com/en/java/>
- OpenAPI Initiative. (2026). *OpenAPI Specification Documentation*.
Especificación utilizada para documentar APIs REST.
Disponible en:
<https://spec.openapis.org/oas/>
- Springdoc. (2026). *Springdoc OpenAPI Documentation*.
Documentación para integración de Swagger/OpenAPI con Spring Boot.
Disponible en:
<https://springdoc.org/>

- JWT. (2026). *Java JWT: JSON Web Token for Java and Android*.
Librería utilizada para generación y manejo de tokens JWT.
Disponible en:
<https://github.com/jwtkt/jjwt>
- Apache Maven. (2026). *Maven Documentation*.
Herramienta utilizada para administración de dependencias y construcción del proyecto.
Disponible en:
<https://maven.apache.org/guides/>
- JUnit Team. (2026). *JUnit 5 User Guide*.
Framework utilizado para pruebas unitarias.
Disponible en:
<https://junit.org/junit5/docs/current/user-guide/>
- Mockito. (2026). *Mockito Documentation*.
Framework utilizado para creación de objetos simulados (mocks) durante pruebas unitarias.
Disponible en:
<https://site.mockito.org/>
- Git. (2026). *Git Documentation*.
Sistema de control de versiones utilizado durante el desarrollo.
Disponible en:
<https://git-scm.com/doc>
- GitHub. (2026). *GitHub Documentation*.
Plataforma utilizada para almacenamiento del repositorio y colaboración entre desarrolladores.
Disponible en:
<https://docs.github.com/>